

UNIVERSITÀ POLITECNICA DELLE MARCHE
FACOLTÀ DI INGEGNERIA
Dipartimento di Ingegneria dell'Informazione
Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione



RELAZIONE DI PROGETTO

Sistema Oracolo Bayesiano per Catena del Freddo Farmaceutica

Bayesian Oracle System for Pharmaceutical Cold Chain

Professore

Luca Spalazzi

Studenti

Luigi Greco

Andrea Altieri

Filippo Marchegiani

Luca Belardinelli

ANNO ACCADEMICO 2025-2026

Indice	i
1 Introduzione	1
1.1 Il problema della Catena del Freddo	1
1.2 Obiettivi del Progetto	2
2 Analisi del rischio	3
2.1 Modellazione i^*	3
2.1.1 Scenario senza sistema: supply chain tradizionale	3
2.1.2 Scenario con sistema: introduzione della blockchain	4
2.1.3 Scenario di minaccia: analisi degli attaccanti	5
2.2 Analisi delle minacce DUAL-STRIDE	6
2.3 Analisi dettagliata degli scenari (<i>Abuse e Misuse Cases</i>)	9
2.3.1 Integrità dei dati IoT (Asset A2)	9
2.3.2 Sicurezza dei pagamenti e fondi (Asset A3)	11
2.3.3 Configurazione e logica bayesiana (Asset A5 e A6)	14
2.3.4 Interfaccia utente e credenziali (Asset A7 e A8)	17
2.4 Standard internazionali e mitigazioni	21
2.5 Conclusioni e rischio residuo	22
3 Analisi e progettazione architetturale	24
3.1 Design Architetturale e Strategie di Sicurezza	24
3.1.1 Architettura distribuita, ridondante e diversificata	24
3.1.2 Interazione utente e strumenti di sviluppo	24
3.1.3 Monitoraggio, isolamento e offuscamento	25
3.2 Design degli asset e linee guida di sicurezza	25
3.3 Modellazione Markov Chain e verifica formale con PRISM	25
3.3.1 Definizione e dinamica del modello	26
Componenti del modello	27
3.3.2 Formalizzazione della matrice di transizione	27
3.3.3 Calcolo della distribuzione di Equilibrio tramite Autovalori	28
3.3.4 Analisi della distribuzione limite	29
3.3.5 Verifica delle proprietà PCTL	29
Analisi di Safety: integrità finanziaria	29
Analisi di Guarantee: resilienza e rimborsi	30

Analisi quantitativa e reward	30
3.4 Monitoring	31
3.4.1 Enforcement della proprietà di safety (S4: Probability Threshold) . . .	31
3.4.2 Enforcement della proprietà di guarantee (G1: Payment Validity) . . .	32
3.4.3 Adozione di librerie standard (OpenZeppelin)	32
3.5 Architettura modulare del frontend	32
3.6 Motivazione delle scelte tecnologiche	33
3.6.1 Analisi di resistenza, ambiguità e sopravvivenza	33
3.6.2 Mitigazione delle debolezze	33
4 Analisi della qualità del codice e testing	34
4.1 Introduzione	34
4.1.1 Solhint nel progetto	34
4.2 Metodologia	34
4.2.1 Funzionamento di Solhint	34
4.2.2 Installazione ed esecuzione	35
4.3 Risultati iniziali	35
4.3.1 Analisi dei warning NatSpec	36
4.4 Analisi architetturale dei contratti	36
4.4.1 BNCore.sol - analisi dettagliata	36
4.4.2 BNGestoreSpedizioni.sol - analisi dettagliata	37
4.4.3 BNPagamenti.sol - analisi dettagliata	38
4.5 Processo di ottimizzazione	39
4.5.1 Miglioramenti strutturali e documentazione	39
4.5.2 Adattamento delle regole e pulizia finale	41
4.6 Configurazione e risultati finali	41
A Analisi delle Performance e Scalabilità	43
A.1 Metodologia di Test e Scenari	43
A.2 Analisi dei Risultati	43

La gestione della catena del freddo (Pharmaceutical Cold Chain) rappresenta una delle sfide più critiche nel settore logistico sanitario. Il trasporto di farmaci termosensibili, come vaccini e insulina, richiede il mantenimento rigoroso di specifici range di temperatura (tipicamente 2°C - 8°C) lungo l'intera filiera. Deviazioni anche minime possono compromettere l'efficacia del prodotto, con conseguenze potenzialmente letali per i pazienti e ingenti danni economici per le aziende.

Attualmente, la trasparenza di questo processo è limitata dall'uso di sistemi centralizzati e trust-based, dove le informazioni sono spesso frammentate, cartacee o custodite in sistemi informatici proprietari. Questo scenario rende difficile ricostruire con certezza la "storia termica" di un lotto e lascia spazio a possibili manipolazioni dei dati per coprire errori logistici o negligenze.

In questo contesto, la tecnologia Blockchain offre un cambio di paradigma fondamentale, passando dalla "fiducia negli attori" alla "fiducia nel protocollo". Attraverso un registro distribuito, immutabile e trasparente, è possibile garantire che ogni misurazione registrata sia autentica e non repudiabile. Tuttavia, la blockchain da sola non può verificare la veridicità del dato fisico prima che venga scritto.

Questo progetto propone una soluzione ibrida che integra Hyperledger Besu, una blockchain permissioned adatta a contesti enterprise, con un sistema di Oracoli Bayesiani. L'approccio innovativo risiede nell'utilizzare l'inferenza probabilistica on-chain per validare la coerenza delle letture multisensoriali (temperatura, timestamp, shock, luce, integrità sigillo) prima di finalizzare la transazione di consegna. In questo modo, il sistema non si limita a registrare i dati, ma agisce come un decisore autonomo capace di accettare o rifiutare un lotto in base a politiche di rischio matematicamente definite.

Questa tesi illustra il design, l'implementazione e la verifica di tale architettura sicura, mettendo in luce come l'integrazione tra DLT (Distributed Ledger Technology) e metodi formali possa elevare gli standard di sicurezza e affidabilità nella logistica farmaceutica 4.0.

1.1 Il problema della Catena del Freddo

La spedizione di medicinali sensibili è un processo ad alto rischio. Secondo l'Organizzazione Mondiale della Sanità (OMS), una percentuale significativa di vaccini viene sprecata ogni anno a causa di interruzioni nella catena del freddo. La criticità principale risiede nella mancanza di visibilità end-to-end, poiché i dati di transito sono spesso disponibili solo a

posteriori, impedendo interventi correttivi tempestivi. A ciò si aggiunge un potenziale conflitto di interessi intrinseco alla filiera: il trasportatore, responsabile del mantenimento della temperatura, coincide spesso con la figura deputata a fornire i dati di monitoraggio, creando un incentivo alla manipolazione in caso di guasti o negligenze. Infine, l'eterogeneità dei sistemi informativi utilizzati da produttori, distributori e farmacie ostacola una comunicazione fluida e in tempo reale, rendendo difficile la ricostruzione precisa della storia termica del prodotto.

1.2 Obiettivi del Progetto

Il sistema proposto mira a risolvere queste problematiche attraverso un approccio integrato che automatizza la fiducia e garantisce l'integrità del dato.

In primo luogo, si punta all'*automazione tramite Smart Contract* per eliminare l'intermediazione umana e burocratica nei processi di verifica e pagamento. Il contratto intelligente agisce come un deposito a garanzia (escrow), sbloccando i fondi al corriere solo se tutte le condizioni di qualità pattuite vengono matematicamente soddisfatte, rimuovendo così l'arbitrarietà dalla fase di liquidazione.

Parallelamente, viene garantita la *sicurezza del dato* assicurando che, una volta acquisita, l'informazione non possa essere alterata. Questo è reso possibile dalla crittografia immutabile della blockchain e dal meccanismo di consenso IBFT 2.0 di Hyperledger Besu, che rendono il registro distribuito resistente a tentativi di manomissione.

Infine, l'obiettivo più ambizioso riguarda la *validazione logica* per garantire che il dato acquisito rifletta fedelmente la realtà. L'Oracolo Bayesiano interviene correlando letture diverse, come le evidenze di temperatura, timestamp, shock, luce e integrità del sigillo, per calcolare la probabilità posteriore di un evento avverso. Il sistema è quindi in grado di distinguere tra semplici falsi positivi dei sensori e reali compromissioni del carico, basando le proprie decisioni su un'analisi probabilistica robusta delle evidenze raccolte piuttosto che su singole soglie deterministiche.

In questo capitolo viene presentata un'analisi approfondita della sicurezza del sistema, condotta attraverso metodologie formali e strutturate. Il percorso logico inizia con la modellazione degli obiettivi e delle dipendenze strategiche tramite il framework i^* (*iStar*), prosegue con la valutazione delle minacce mediante l'approccio *DUAL-STRIDE* esteso agli asset dell'attore sistema, e si conclude con la definizione dettagliata di scenari di *Abuse e Misuse Cases*.

2.1 Modellazione i^*

Per comprendere appieno le dinamiche organizzative e di sicurezza, abbiamo adottato il framework i^* (*iStar*), che permette di modellare non solo gli aspetti funzionali ma anche le intenzioni, le dipendenze strategiche e le vulnerabilità sociali del sistema.

L'analisi si articola in tre scenari evolutivi: lo stato attuale, senza sistema; lo stato futuro con l'introduzione della blockchain e, infine, uno scenario di minaccia che modella esplicitamente gli attaccanti.

2.1.1 Scenario senza sistema: supply chain tradizionale

Nel modello iniziale, raffigurato in Figura 2.1, osserviamo le relazioni tra i tre attori principali della filiera: il *Mittente* (farmacia o produttore), il *Corriere* e il sistema di sensoristica isolata (*Sensore IoT*).

In questo scenario, il *Mittente* dipende quasi interamente dal *Corriere* per il raggiungimento dei suoi obiettivi critici. Il goal primario "Spedire farmaci in catena del freddo" si decompone in task operativi come la selezione del vettore e la preparazione dell'imballaggio. Tuttavia, la criticità emerge nella dipendenza per la risorsa "Specifiche spedizione": il mittente deve fidarsi che il corriere rispetti le condizioni pattuite (temperatura, integrità, scadenze).

Il *Corriere*, a sua volta, ha l'obiettivo di "Effettuare consegna" e "Dimostrare integrità del pacco". Qui sorge un conflitto di interessi strutturale: per ottenere lo sblocco del pagamento (goal "Ottenere sblocco pagamento"), il corriere è l'unico certificatore della propria qualità. Sebbene esistano sensori IoT, questi non sono integrati in un sistema inviolabile; pertanto, il mittente non ha garanzie matematiche che i report delle evidenze ("Report evidenze") non siano stati manipolati o omessi.

Il softgoal "Protezione finanziaria" del mittente è quindi costantemente a rischio, poichè dipende da una fiducia cieca verso l'operato logistico. Non esiste un meccanismo di enforce-

ment automatico per eventuali rimborsi in caso di non conformità ("Ottenere rimborso per non-conformità").

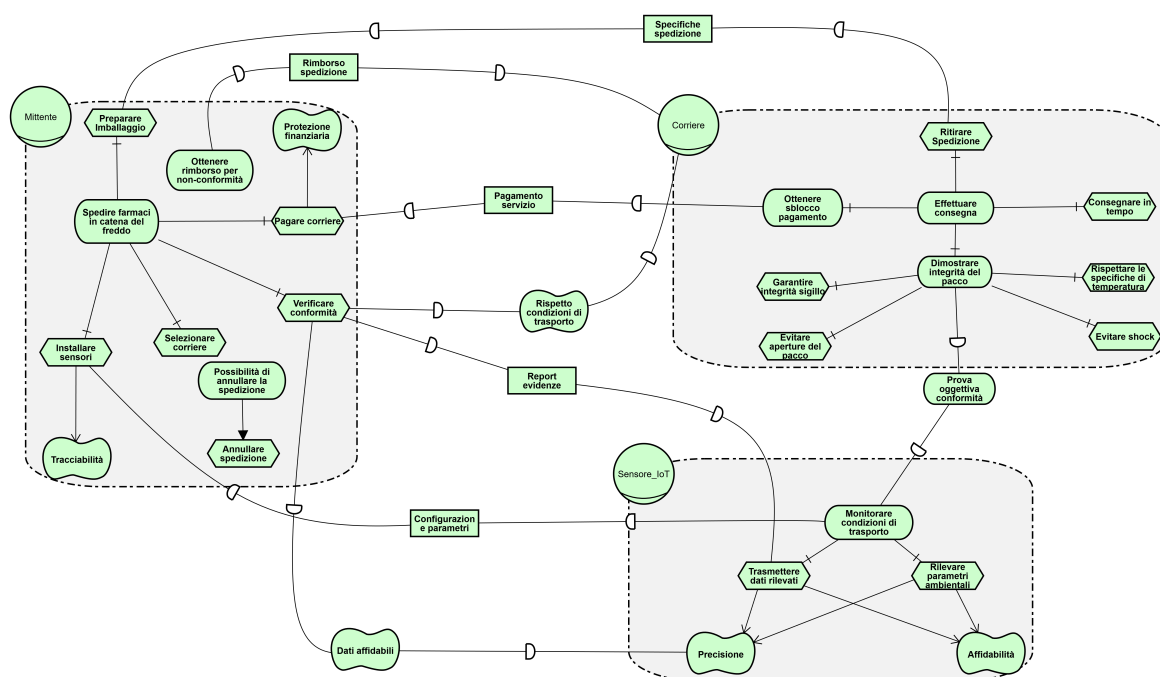


Figura 2.1: Modello SD: supply chain senza sistema

2.1.2 Scenario con sistema: introduzione della blockchain

L'introduzione dello Smart Contract (raffigurato come attore *Sistema*) ridefinisce le dipendenze strategiche, centralizzando la fiducia in un'entità imparziale e automatizzata. Come emerge dal modello, il *Sistema* funge da intermediario tra mittente, corriere e sensori, garantendo che le interazioni avvengano secondo regole certe.

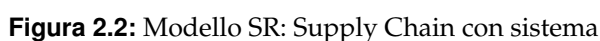
Il *mittente* non dipende più dalla parola del corriere, ma si affida al sistema per una serie di obiettivi cruciali:

- *Validazione evidenze e Pagamento automatico*: il mittente delega al contratto la verifica dei dati e l'esecuzione condizionale del pagamento.
- *Tracciabilità immutabile*: un softgoal soddisfatto intrinsecamente dalla tecnologia blockchain.
- *Rimborso automatico*: in caso di violazione delle condizioni contrattuali, il sistema garantisce il ritorno dei fondi depositati senza necessità di arbitrato esterno.

Il *Corriere* dipende dal sistema per la "Ricezione Pagamento", che avviene solo a fronte della "Conferma Consegna" e della verifica delle condizioni ambientali. Non esiste più discrezionalità umana nel rilascio dei fondi.

Un ruolo chiave è svolto dall'attore *Sensore*, che interagisce direttamente con il sistema fornendo le "Letture Ambientali". In cambio, il sistema fornisce un "Timestamp Certificato", assicurando che ogni dato registrato sia temporalmente collocato in modo inalterabile.

Infine, l'attore *Amministratore* gestisce la governance tecnica, occupandosi della "Configurazione Sistema" (es. parametri bayesiani) e della "Gestione Ruoli", ricevendo in cambio dal sistema la garanzia di "Dati Affidabili".



Nonostante i miglioramenti, il sistema espone nuove superfici di attacco. La Figura 2.3 modella tre profili di avversari che minacciano gli asset del sistema: l'*Attaccante Interno*, l'*Attaccante Esterno* e l'*Utente Maldestro*.

L'*Attaccante Esterno* punta a "Compromettere sistema" sfruttando vulnerabilità tecniche e umane. I suoi vettori di attacco principali includono:

- Infine, l'*Utente Maldestro* rappresenta una minaccia non intenzionale ma altrettanto dannosa. Il suo comportamento è guidato dall'obiettivo negativo di "Errore operativo". Questo include errori di configurazione ("Errore di battitura numerico", "Overflow non gestito"), errori nei pagamenti ("Importo errato", "Destinatario errato") e negligenza nella gestione

delle chiavi private ("Screenshot seed", "Caricamento su cloud pubblico"). Questi "Misuse Cases" evidenziano come la sicurezza non dipenda solo dalla robustezza del codice, ma anche dall'usabilità e dalla formazione degli operatori.

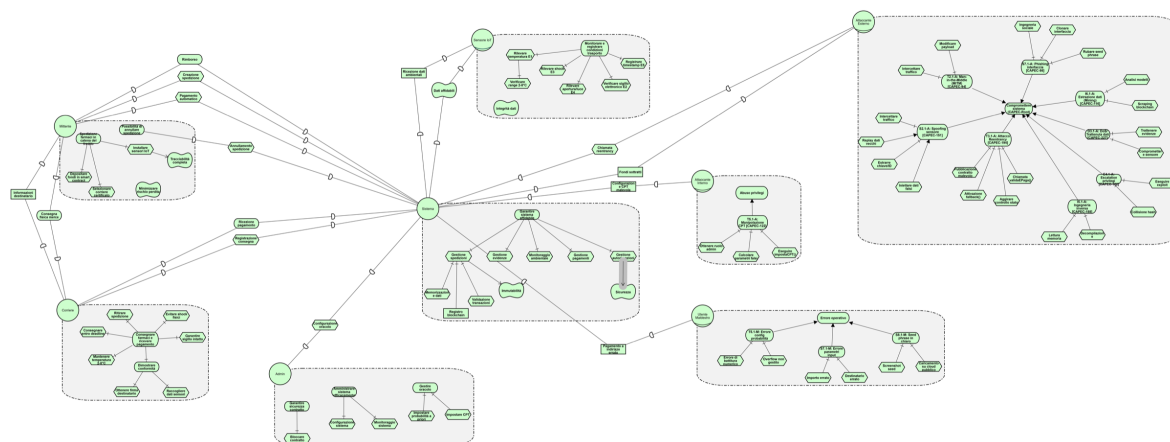


Figura 2.3: Modello SR: analisi degli attaccanti e alberi di attacco

2.2 Analisi delle minacce DUAL-STRIDE

L'analisi è stata condotta raggruppando gli asset secondo il paradigma DUAL-STRIDE, che estende il classico STRIDE includendo aspetti di affidabilità e resilienza (DUAL).

Prima di esaminare le minacce, è essenziale definire gli asset critici del sistema che necessitano di protezione. La Tabella 2.1 elenca le componenti primarie, spaziando dalla logica on-chain ai dispositivi fisici.

Tabella 2.1: Asset critici del sistema

<i>ID</i>	<i>Asset</i>	<i>Descrizione</i>	<i>Criticità</i>
A1	Smart Contract	logica di business e fondi in escrow	<i>Molto alta</i>
A2	Evidenze IoT	dati dai sensori (E1-E5)	<i>Molto alta</i>
A3	Pagamenti ETH	fondi depositati dai mittenti	<i>Molto alta</i>
A4	Ruoli e permessi	sistema di controllo degli accessi	<i>Alta</i>
A5	CPT e probabilità	parametri della rete bayesiana	<i>Alta</i>
A6	Dati spedizioni	record on-chain e storico	<i>Media</i>
A7	Interfaccia Web	frontend utente	<i>Media</i>
A8	Chiavi private	credenziali MetaMask	<i>Molto alta</i>

La tabella successiva offre una visione d'insieme delle minacce identificate per ciascun asset, mappando le categorie STRIDE/DUAL sulle contromisure implementate.

Riepilogo visuale minacce (DUAL-STRIDE)

Asset	Valore	Spoofing	Tampering	Repudiation	Information disclosure	Denial of service	Elevation of privilege	Danger	Unreliability	Absence of resilience	Leakage	Descrizione attacco	Mitigazione
A1	Critico		•					•				T1.1: manipolazione logica codice compilato	Analisi + Verifica formale
A1	Critico					•				•		D1.1: blocco contratto (saturazione dello storage)	Limiti gas
A1	Critico						•				•	E1.1: escalation privilegi amministratore	Est. Gnosis Safe multi-firma
A2	Critico	•						•				S2.1: impersonificazione sensore	Controllo accessi per ruoli
A2	Critico		•					•				T2.1: modifica evidenze in transito	Firma transazione (autenticazione mittente)
A2	Critico			•								R2.1: ripudio invio evidenza	Registrazione eventi
A2	Critico				•				•			I2.1: corruzione dati sensore	Ridondanza E1-E5
A3	Critico		•					•				T3.1: attacco reentrancy	ReentrancyGuard
A3	Critico					•				•		D3.1: blocco fondi escrow	Logica rimborsi + tempo massimo
A3	Critico						•				•	E3.1: drenaggio fondi contratto	Pagamento sicuro + ReentrancyGuard
A4	Alto	•						•				S4.1: assegnazione ruolo fraudolenta	Concessione solo amministratore
A4	Alto		•					•				T4.1: modifica mapping ruoli	Variabili di stato private
A4	Alto						•				•	E4.1: escalation privilegi	Controlli multilivello
A5	Alto		•					•				T5.1: manipolazione CPT	Variabili private + amministratore
A5	Alto				•						•	I5.1: analisi inversa CPT	Offuscamento
A5	Alto						•				•	E5.1: fuga parametri bayesiani	Funzioni di lettura private
A6	Medio				•						•	I6.1: estrazione dati competitiva	Offuscamento dati sensibili
A6	Medio		•									T6.1: modifica storico spedizioni	Immutabilità blockchain
A6	Medio			•								R6.1: ripudio transazione	Log eventi permanenti
A7	Medio	•						•				S7.1: phishing/impersonificazione interfaccia	Formazione utente + HTTPS
A7	Medio		•					•				T7.1: attacco sulla connessione Web3	TLS + SRI
A7	Medio				•						•	I7.1: iniezione di codice (XSS)	Bonifica degli input

<i>Asset</i>	<i>Valore</i>	<i>S</i> Spoofing	<i>T</i> Tampering	<i>R</i> Reputation	<i>I</i> Information Disclosure	<i>D</i> DoS	<i>E</i> Elevation of privilege	<i>D</i> Danger	<i>U</i> Unreliability	<i>A</i> Absence of resilience	<i>L</i> Leakage/Exp.	<i>Descrizione attacco</i>	<i>Mitigazione (Controllo)</i>
A8	Critico	•						•				S8.1: furto chiavi (captatore di digitazioni)	Portafoglio hardware (hardware wallet)
A8	Critico				•						•	I8.1: esposizione frase di recupero	Formazione per l'archiviazione sicura
A8	Critico						•				•	E8.1: accesso non autorizzato al portafoglio	Politica password + 2FA

2.3 Analisi dettagliata degli scenari (*Abuse e Misuse Cases*)

A completamento dell'analisi tabellare, approfondiamo gli scenari più significativi distinguendo tra *Abuse Cases* (azioni malevole intenzionali) e *Misuse Cases* (errori non intenzionali o guasti), organizzati per area tematica.

2.3.1 Integrità dei dati IoT (Asset A2)

La validità del sistema dipende interamente dalla veridicità dei dati trasmessi dai sensori. Qui analizziamo il rischio di impersonificazione da parte di un attaccante esterno e l'inaffidabilità dovuta a guasti hardware.

ABUSE CASE: S2.1-A

Campo	Contenuto
Tipologia scenario	Abuse Case
Caso d'uso	invio evidenza sensore
ID scenario	S2.1-A
Nome scenario	<i>sensore malevolo falsificato</i>
Attori	attaccante esterno con chiave compromessa (ruolo sensore)
Descrizione	un attaccante riesce ad ottenere la chiave privata di un account autorizzato (ad esempio tramite phishing, software malevolo o compromissione fisica del dispositivo) e la sfrutta per inviare evidenze falsificate allo smart contract, facendo sì che spedizioni non conformi vengano erroneamente validate.
Dati	chiave privata del sensore, ID spedizione bersaglio, valori evidenze falsificati (E1-E5)
Precondizioni	- spedizione in stato "In Attesa" - l'attaccante possiede una chiave autorizzata - bersaglio: spedizione con merce deteriorata (temperatura fuori limite)
Flusso principale	1. l'attaccante individua la spedizione monitorando gli eventi sulla blockchain 2. invia evidenze inserendo valori positivi fittizi 3. ripete l'operazione per tutti i sensori coinvolti 4. il corriere richiede il pagamento e il sistema valida la spedizione 5. il corriere viene pagato illegittimamente per merce non conforme
Flusso alternativo	- se la transazione viene rifiutata per mancanza di autorizzazioni, l'attacco fallisce - se l'evidenza è già registrata, l'attaccante tenta con un altro account compromesso
Eccezioni	- il sistema rileva invii troppo ravvicinati nel tempo (anomalia temporale) - tentativi multipli dallo stesso sensore vengono bloccati automaticamente

<i>Risultato</i>	<i>se l'attacco riesce</i> : il mittente paga per merce deteriorata <i>tracciabilità</i> : ogni operazione resta registrata sulla blockchain per analisi future
<i>Commenti</i>	CAPEC-151: Impersonificazione (Identity Spoofing) <i>contromisure consigliate</i> : rotazione periodica delle chiavi, vincolo hardware per il dispositivo, autenticazione a sfida

MISUSE CASE: S2.1-M

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Misuse Case
<i>Caso d'uso</i>	invio evidenza sensore
<i>ID scenario</i>	S2.1-M
<i>Nome scenario</i>	<i>guasto hardware sensore (Inaffidabilità)</i>
<i>Attori</i>	sensore IoT difettoso o non calibrato
<i>Descrizione</i>	un sensore legittimo trasmette dati errati a causa di problemi hardware: perdita di calibrazione, batteria scarica o interferenze elettromagnetiche. Ciò può generare valutazioni errate della spedizione senza intenzione malevola.
<i>Dati</i>	letture sensore errate, marca temporale
<i>Precondizioni</i>	- sensore in uso da lungo tempo senza manutenzione - ambiente con forti interferenze - livello batteria critico
<i>Flusso principale</i>	1. il sensore di temperatura presenta un errore di misura 2. una spedizione conforme viene letta come fuori limite 3. il sensore invia una lettura negativa erroneamente 4. la rete bayesiana calcola una probabilità di conformità troppo bassa 5. la validazione fallisce e il mittente perde il deposito cauzionale
<i>Flusso alternativo</i>	- gli altri sensori compensano l'errore grazie alla ridondanza bayesiana - allo scadere del tempo massimo, il mittente richiede il rimborso automatico
<i>Eccezioni</i>	- il sistema rileva incongruenze macroscopiche tra i dati dei vari sensori - intervento manuale dell'amministratore per sbloccare la situazione
<i>Risultato</i>	<i>impatto</i> : rifiuto ingiusto di merce conforme con perdita economica per il mittente <i>requisiti non funzionali</i> : manutenzione periodica, monitoraggio della batteria
<i>Commenti</i>	DUA-Unreliability: l'assenza di manutenzione è la causa principale <i>contromisure consigliate</i> : algoritmi per il rilevamento della deriva dei dati, confronto tra sensori multipli

2.3.2 Sicurezza dei pagamenti e fondi (Asset A3)

Gli smart contract gestiscono valore economico reale, rendendoli bersagli primari. Esaminiamo il classico attacco di reentrancy e, parallelamente, la possibilità di errori umani o blocchi involontari dei fondi (DoS).

ABUSE CASE: T3.1-A

Campo	Contenuto
Tipologia scenario	Abuse Case
Caso d'uso	pagamento mittente
ID scenario	T3.1-A
Nome scenario	<i>reentrancy attack su validaEPaga</i>
Attori	attaccante che pubblica uno smart contract malevolo registrato come corriere
Descrizione	l'attaccante pubblica un contratto malevolo dotato di una funzione di ricezione che richiama automaticamente la validazione. In assenza di protezioni, quando il sistema esegue il trasferimento dei fondi, il contratto malevolo rientra nella funzione prima dell'aggiornamento dello stato, consentendo prelievi multipli per la stessa spedizione.
Dati	contratto corriere malevolo, ID spedizione, saldo complessivo dello smart contract
Precondizioni	- l'attaccante crea una spedizione usando il proprio contratto malevolo come indirizzo del corriere - la spedizione ha evidenze complete e valide - lo smart contract detiene fondi in escrow di più spedizioni attive
Flusso principale	1. l'attaccante pubblica un contratto con funzione di ricezione malevola 2. una spedizione viene creata con tale contratto come corriere 3. le evidenze risultano valide 4. il sistema trasferisce i fondi al contratto malevolo, la cui funzione <code>receive</code> richiama automaticamente <code>validaEPaga</code> prima che lo stato venga aggiornato 5. il ciclo si ripete drenando i fondi nel contratto
Flusso alternativo	- se la protezione contro la reentrancy è attiva, la seconda chiamata viene annullata - se si segue il corretto ordine operativo (aggiornamento stato prima dell'invio), la rientranza è impedita
Eccezioni	- il ciclo si interrompe per esaurimento del gas per la transazione - il saldo del contratto diventa insufficiente e l'operazione fallisce
Risultato	<i>se l'attacco riesce:</i> tutti i fondi nel contratto vengono sottratti <i>se l'attacco fallisce:</i> la transazione viene annullata e i fondi restano al sicuro
Commenti	CAPEC-194: Iniezione di risorse fittizie contromisure implementate: ReentrancyGuard, pattern Checks-Effects-Interactions (CEI), pagamenti diretti protetti

MISUSE CASE: T3.1-M

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Misuse Case
<i>Caso d'uso</i>	creazione spedizione
<i>ID scenario</i>	T3.1-M
<i>Nome scenario</i>	<i>errore indirizzo corriere (Errore utente)</i>
<i>Attori</i>	mittente distratto
<i>Descrizione</i>	il mittente inserisce per errore un indirizzo sbagliato nel campo destinato al corriere. Poiché la blockchain non verifica l'identità reale dietro un indirizzo, il pagamento finale verrà inviato al destinatario errato senza possibilità di recupero.
<i>Dati</i>	indirizzo corriere errato, fondi ETH depositati
<i>Precondizioni</i>	- creazione della spedizione tramite l'interfaccia web - inserimento manuale dell'indirizzo - assenza di verifica dell'identità del destinatario
<i>Flusso principale</i>	1. il mittente digita erroneamente l'indirizzo del corriere 2. la spedizione procede regolarmente con l'invio delle evidenze 3. la validazione trasferisce i fondi all'indirizzo digitato 4. il corriere legittimo non riceve il compenso
<i>Flusso alternativo</i>	- se l'indirizzo non esiste o è malformato, la transazione potrebbe fallire preventivamente
<i>Eccezioni</i>	- l'uso di una rubrica di indirizzi certificati previene l'errore di digitazione
<i>Risultato</i>	<i>impatto</i> : perdita definitiva dei fondi e possibile contenzioso legale <i>requisiti non funzionali</i> : interfaccia con selezione da elenco di operatori verificati
<i>Commenti</i>	<i>DUA-Exposure</i> : l'interfaccia espone l'utente a errori umani onerosi <i>contromisure consigliate</i> : lista bianca di corrieri, rubrica integrata nell'applicazione

ABUSE CASE: D3.1-A

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Abuse Case
<i>Caso d'uso</i>	validazione spedizione
<i>ID scenario</i>	D3.1-A
<i>Nome scenario</i>	<i>blocco intenzionale delle evidenze (Denial of Service)</i>
<i>Attori</i>	seniore compromesso o corriere collusivo
<i>Descrizione</i>	un attaccante che controlla un sensore o un corriere rifiuta intenzionalmente di inviare l'ultima evidenza necessaria, bloccando i fondi del mittente in escrow come forma di estorsione o sabotaggio.
<i>Dati</i>	ID spedizione, evidenze parziali inviate, ultima evidenza trattenuta
<i>Precondizioni</i>	- spedizione in attesa di completamento - gran parte delle evidenze sono già registrate - l'attaccante ha il controllo del caricamento dati residuo

<i>Flusso principale</i>	1. la spedizione procede regolarmente fino alle fasi finali 2. l'attaccante trattiene deliberatamente l'ultimo dato necessario 3. senza il set completo, la validazione non può essere completata 4. i fondi del mittente restano bloccati nel contratto a tempo indeterminato
<i>Flusso alternativo</i>	- l'attaccante richiede un compenso extra per l'invio del dato mancante - manomissione simultanea di più canali di invio per rendere il blocco efficace
<i>Eccezioni</i>	- allo scadere del tempo massimo prefissato, il mittente può richiedere il rimborso automatico - intervento manuale dell'amministratore per sbloccare i fondi
<i>Risultato</i>	<i>se l'attacco riesce:</i> blocco temporaneo dei fondi e danno reputazionale al sistema <i>contromisura attiva:</i> rimborso automatico garantito dopo il periodo di inattività
<i>Commenti</i>	CAPEC-603: Blockage <i>contromisure:</i> tempo massimo (timeout) con logica di rimborso dopo tentativi falliti

MISUSE CASE: D3.1-M

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Misuse Case
<i>Caso d'uso</i>	validazione spedizione
<i>ID scenario</i>	D3.1-M
<i>Nome scenario</i>	<i>perdita connettività IoT (Assenza di resilienza)</i>
<i>Attori</i>	seniore offline per guasto di rete o alimentazione
<i>Descrizione</i>	un sensore perde la connessione durante il trasporto (batteria scarica, assenza di campo o guasto hardware) e non trasmette le evidenze, causando il blocco involontario dei fondi del mittente.
<i>Dati</i>	evidenze parziali, log di connessione del sensore
<i>Precondizioni</i>	- spedizione in transito in zone con scarsa copertura - livello batteria del sensore ormai critico - assenza di memoria interna per invii differiti
<i>Flusso principale</i>	1. la spedizione parte regolarmente 2. durante il tragitto si perde il segnale di rete 3. solo una parte delle evidenze viene trasmessa correttamente 4. la batteria del sensore si esaurisce prima del ripristino della connettività 5. la merce arriva ma il sistema non può validare la spedizione
<i>Flusso alternativo</i>	- il sensore riaggancia la rete in tempo per terminare l'invio - recupero manuale dei dati dalla memoria interna del dispositivo
<i>Eccezioni</i>	- il sistema permette il rimborso automatico al mittente dopo 7 giorni di inattività - il corriere fornisce prove alternative della conformità

<i>Risultato</i>	<i>impatto</i> : degradazione del servizio con attesa forzata per il rimborso <i>requisiti non funzionali</i> : garanzia di disponibilità, batterie di lunga durata, doppia SIM
------------------	--

2.3.3 Configurazione e logica bayesiana (Asset A5 e A6)

Questi scenari si concentrano sulla manipolazione della logica decisionale del sistema (le tabelle CPT) e sull'estrazione di informazioni sensibili tramite analisi inversa o analisi dei dati pubblici.

ABUSE CASE: T5.1-A

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Abuse Case
<i>Caso d'uso</i>	configurazione rete bayesiana
<i>ID scenario</i>	T5.1-A
<i>Nome scenario</i>	<i>abuso privilegi amministratore sui parametri bayesiani</i>
<i>Attori</i>	amministratore infedele
<i>Descrizione</i>	un amministratore con privilegi elevati modifica intenzionalmente le tabelle di probabilità condizionate (CPT) per alterare la logica del sistema, favorendo validazioni fraudolente o sabotando il servizio.
<i>Dati</i>	parametri delle tabelle CPT, chiavi di accesso dell'amministratore
<i>Precondizioni</i>	- amministratore compromesso o intenzionalmente ostile - parametri modificabili tramite funzioni dedicate - assenza di monitoraggio sulle modifiche effettuate
<i>Flusso principale</i>	1. l'amministratore aggiorna le tabelle CPT con valori arbitrari 2. imposta una probabilità di esito positivo elevata anche per parametri di freschezza negativi 3. ogni spedizione, a prescindere dallo stato reale, supera la validazione 4. pagamento indebito a soggetti collusivi 5. compromissione totale della credibilità del sistema di calcolo
<i>Flusso alternativo</i>	- l'amministratore imposta parametri tali da impedire sistematicamente i pagamenti legittimi
<i>Eccezioni</i>	- un sistema di monitoraggio rileva tassi di approvazione anomali e richiede verifiche - l'uso di una gestione multi-firma impedisce a un singolo individuo di modificare la logica
<i>Risultato</i>	<i>se l'attacco riesce</i> : perdita di integrità logica e danni economici diffusi <i>rilevamento</i> : analisi statistica delle prestazioni del sistema e allarmi su anomalie
<i>Commenti</i>	CAPEC-122: Abuso di privilegi <i>contromisure consigliate</i> : controllo multi-firma per le modifiche, parametri bloccati dopo la fase di calibrazione

MISUSE CASE: T5.1-M

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Misuse Case
<i>Caso d'uso</i>	configurazione rete bayesiana
<i>ID scenario</i>	T5.1-M
<i>Nome scenario</i>	<i>errore configurazione probabilità (Errore umano)</i>
<i>Attori</i>	amministratore inesperto o distratto
<i>Descrizione</i>	un amministratore commette un errore non intenzionale nell'inserimento dei valori delle CPT: inversione di cifre, valori fuori scala o interpretazione errata delle probabilità condizionate.
<i>Dati</i>	parametri CPT errati, registrazione della transazione di modifica
<i>Precondizioni</i>	- aggiornamento del modello bayesiano in corso - validazione dei range presente ma non sufficiente per errori logici - assenza di un ambiente di prova (staging)
<i>Flusso principale</i>	1. l'operatore aggiorna le CPT con nuovi valori 2. digita per errore un valore coerente ma scorretto (es. 51% invece di 15%) 3. il sistema accetta il dato poiché rientra nel range ammesso 4. i calcoli producono risultati distorti senza che nessuno se ne accorga 5. spedizioni conformi vengono rifiutate (o viceversa) fino alla correzione
<i>Flusso alternativo</i>	- l'operatore digita un valore fuori scala (es. 150%) e il sistema blocca automaticamente l'operazione - errore nella disposizione delle probabilità (inversione della condizionalità)
<i>Eccezioni</i>	- vincoli rigorosi sui valori ammessi integrati nel codice del contratto - test preventivi in ambiente di simulazione prima dell'aggiornamento reale
<i>Risultato</i>	<i>impatto</i> : instabilità del sistema con decisioni errate fino al ripristino dei valori corretti <i>requisiti non funzionali</i> : validazione rigorosa degli input e rilascio controllato
<i>Commenti</i>	<i>DUA-Unreliability</i> : l'interfaccia non impedisce errori semantici nei parametri <i>contromisure consigliate</i> : ambiente di staging pre-produzione, verifica incrociata dei valori inseriti, log immutabile delle modifiche

ABUSE CASE: I6.1-A

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Abuse Case
<i>Caso d'uso</i>	consultazione storico spedizioni
<i>ID scenario</i>	I6.1-A

Nome scenario	<i>analisi competitiva dei dati pubblici</i>
Attori	azienda concorrente o analista dati esterno
Descrizione	un concorrente analizza le transazioni pubbliche sulla blockchain per estrarre informazioni di mercato strategiche: volumi di vendita, rotte commerciali, fedeltà dei clienti e stima dei margini di profitto.
Dati	eventi di creazione e pagamento spedizioni, importi in valuta, indirizzi dei partecipanti
Precondizioni	- blockchain pubblica con log degli eventi accessibili - eventi non offuscati o crittografati - associazione degli indirizzi a identità reali tramite informazioni esterne
Flusso principale	1. il concorrente raccoglie tutti gli eventi registrati dal sistema 2. identifica i clienti principali tramite gli indirizzi dei mittenti 3. analizza le tempistiche per rilevare picchi stagionali della domanda 4. stima i volumi d'affari correlando gli importi dei pagamenti 5. utilizza queste informazioni per sottrarre clienti o definire strategie di prezzo aggressive
Flusso alternativo	- analisi dei pattern logistici per ottimizzare le proprie rotte a scapito altrui - studio dei fallimenti per individuare debolezze operative del sistema
Eccezioni	- l'uso di tecniche di oscuramento dei dati sensibili limita l'estrazione di informazioni utili - meccanismi di prova a conoscenza zero nascondono i dettagli finanziari
Risultato	<i>impatto</i>: erosione del vantaggio competitivo ed esposizione di segreti industriali <i>aspetto legale</i>: possibile violazione delle norme sul segreto commerciale
Commenti	CAPEC-169: Footprinting <i>contromisure consigliate</i>: offuscamento dei parametri sensibili, restrizione degli accessi ai log, transazioni private

ABUSE CASE: I5.1-A

Campo	Contenuto
Tipologia scenario	Abuse Case
Caso d'uso	lettura parametri bayesiani
ID scenario	I5.1-A
Nome scenario	<i>analisi inversa dello stato del contratto</i>
Attori	attaccante o concorrente esperto
Descrizione	sebbene alcune variabili siano dichiarate private, un attaccante può leggere direttamente lo stato della memoria del contratto tramite chiamate di basso livello, ricostruendo l'intera struttura della rete bayesiana.

<i>Dati</i>	codice compilato del contratto, layout della memoria, valori delle probabilità
<i>Precondizioni</i>	- contratto pubblicato su blockchain accessibile - dati sensibili memorizzati in chiaro nella memoria operativa del contratto
<i>Flusso principale</i>	1. l'attaccante individua l'indirizzo del contratto 2. utilizza strumenti di scansione della memoria per ogni registro occupato 3. decodifica la struttura dei dati relativi alle probabilità CPT 4. analizza il codice per comprendere la logica decisionale completa 5. ricrea un sistema identico basato sulla stessa logica (furto intellettuale)
<i>Flusso alternativo</i>	- l'attaccante correla i parametri estratti con dati pubblici sulle spedizioni per ricostruire le logiche commerciali - analisi automatizzata con script di decodifica dello storage EVM
<i>Eccezioni</i>	- i parametri vengono calcolati off-chain e solo l'hash del risultato è salvato on-chain - l'uso di tecniche di offuscamento rende la decodifica impraticabile in tempi utili
<i>Risultato</i>	<i>impatto</i> : furto del modello bayesiano proprietario e perdita di esclusività tecnologica
<i>Commenti</i>	CAPEC-188: Ingegneria inversa <i>contromisure consigliate</i> : calcoli fuori blocchi (off-chain) con prove di validità, crittografia dei parametri sensibili

2.3.4 Interfaccia utente e credenziali (Asset A7 e A8)

Infine, analizziamo il vettore di attacco più comune: il fattore umano. Dagli attacchi di phishing (impersonificazione dell'interfaccia) al furto o smarrimento delle chiavi private, questi scenari eludono spesso le difese crittografiche del contratto.

ABUSE CASE: S7.1-A

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Abuse Case
<i>Caso d'uso</i>	accesso applicazione (DApp)
<i>ID scenario</i>	S7.1-A
<i>Nome scenario</i>	<i>clonazione dell'interfaccia via phishing</i>
<i>Attori</i>	attaccante (phisher), utente vittima
<i>Descrizione</i>	l'attaccante crea una replica identica dell'interfaccia legittima e la ospita su un dominio dal nome simile per ingannare l'utente. L'obiettivo è sottrarre la frase di recupero o indurlo a firmare transazioni malevole.
<i>Dati</i>	frase di recupero, chiavi private, permessi di accesso
<i>Precondizioni</i>	- l'utente riceve un collegamento ingannevole - l'interfaccia non è verificata - l'utente non controlla l'indirizzo nel browser

<i>Flusso principale</i>	1. l'attaccante registra un dominio con nome quasi identico all'originale 2. vi pubblica una copia dell'interfaccia con un sistema di cattura dati 3. invia comunicazioni fraudolente richiedendo un aggiornamento 4. l'utente accede al sito fasullo 5. viene richiesto di inserire la frase di recupero per riconnettere il portafoglio 6. l'attaccante sottrae i dati e svuota il portafoglio
<i>Flusso alternativo</i>	- richiesta di autorizzazione fraudolenta per lo spostamento dei fondi - iniezione di codice malevolo tramite canali di distribuzione compromessi
<i>Eccezioni</i>	- l'utente riconosce il dominio fraudolento e segnala il tentativo - il browser mostra un avviso di certificato non valido per il sito clonato
<i>Risultato</i>	<i>se l'attacco riesce:</i> furto completo dei fondi e compromissione dell'identità <i>requisiti non funzionali:</i> formazione sulla sicurezza, connessione sicura obbligatoria
<i>Commenti</i>	CAPEC-98: Phishing <i>contromisure consigliate:</i> uso di protocolli sicuri, verifica del nome tramite servizi di identità, avvisi di sicurezza integrati

MISUSE CASE: S7.1-M

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Misuse Case
<i>Caso d'uso</i>	creazione spedizione
<i>ID scenario</i>	S7.1-M
<i>Nome scenario</i>	<i>errore di inserimento parametri (Errore utente)</i>
<i>Attori</i>	mittente distratto
<i>Descrizione</i>	un utente commette un errore durante l'inserimento dei dati nell'interfaccia autentica: indirizzo errato, importo sbagliato o inversione dei parametri di spedizione.
<i>Dati</i>	parametri della transazione errati, costi per il gas
<i>Precondizioni</i>	- moduli con campi a inserimento libero - validazione lato utente non esaustiva - errore durante il copia-incolla delle informazioni
<i>Flusso principale</i>	1. il mittente inserisce un importo superiore a quello previsto per errore 2. conferma l'operazione senza rileggere i dettagli 3. la transazione viene eseguita con l'importo errato 4. in caso di validazione positiva, il corriere riceve un compenso eccessivo 5. il mittente perde la differenza senza possibilità di annullamento

<i>Flusso alternativo</i>	- l'utente si accorge dell'errore prima della conferma e annulla l'operazione - l'importo errato supera il saldo disponibile e la transazione fallisce
<i>Eccezioni</i>	- l'interfaccia mostra un riepilogo dettagliato con richiesta di conferma esplicita - vincoli di importo minimo/massimo impediscono valori palesemente errati
<i>Risultato</i>	<i>impatto</i> : perdita economica immediata per l'utente <i>requisiti non funzionali</i> : validazione avanzata nell'interfaccia e finestre di conferma dettagliate
<i>Commenti</i>	<i>DUA-Exposure</i> : un'interfaccia poco intuitiva espone a errori costosi <i>contromisure consigliate</i> : vincoli sugli input, anteprima della transazione prima della firma definitiva

ABUSE CASE: S8.1-A

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Abuse Case
<i>Caso d'uso</i>	gestione portafoglio
<i>ID scenario</i>	S8.1-A
<i>Nome scenario</i>	sottrazione credenziali tramite software malevolo
<i>Attori</i>	attaccante con software malevolo, vittima
<i>Descrizione</i>	l'attaccante utilizza un captatore di digitazioni o un sistema di monitoraggio degli appunti sul dispositivo della vittima per catturare la frase di recupero o le chiavi private.
<i>Dati</i>	frase di recupero, file delle chiavi, password del portafoglio
<i>Precondizioni</i>	- l'utente installa inconsapevolmente software compromesso - sistema di monitoraggio attivo in background - l'utente esegue operazioni sensibili sul portafoglio
<i>Flusso principale</i>	1. l'utente scarica un'applicazione apparentemente innocua ma malevola 2. il software installa un captatore di digitazioni 3. l'utente inserisce la frase di recupero per ripristinare l'identità 4. i dati vengono inviati all'attaccante 5. l'attaccante importa le credenziali e svuota i fondi immediatamente
<i>Flusso alternativo</i>	- l'attaccante utilizza un sistema di monitoraggio degli appunti anziché un captatore di digitazioni - compromissione del backup automatico del dispositivo per estrarre i dati
<i>Eccezioni</i>	- un software antivirus rileva e blocca il captatore di digitazioni - l'uso di un portafoglio hardware rende inutile la cattura della password software

<i>Risultato</i>	<i>se l'attacco riesce: furto totale dei fondi e perdita permanente dell'identità</i> <i>rilevamento: spesso possibile solo a furto avvenuto</i>
<i>Commenti</i>	<i>ATT&CK-T1056.001: Captazione di digitazioni</i> <i>contromisure consigliate: uso obbligatorio di portafogli hardware, protezione attiva del sistema operativo</i>

MISUSE CASE: S8.1-M

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Misuse Case
<i>Caso d'uso</i>	salvataggio delle credenziali
<i>ID scenario</i>	S8.1-M
<i>Nome scenario</i>	<i>frase di recupero salvata in chiaro (Negligenza)</i>
<i>Attori</i>	utente inesperto o negligente
<i>Descrizione</i>	l'utente salva la frase di recupero in formati non sicuri: immagini nel cloud, messaggi non protetti o note non cifrate.
<i>Dati</i>	frase di recupero in chiaro, copie di backup online
<i>Precondizioni</i>	- utente poco consapevole dei rischi - backup automatici attivi nel cloud - assenza di strumenti per la gestione sicura dei segreti
<i>Flusso principale</i>	1. alla creazione del portafoglio, l'utente scatta una foto alla frase di recupero 2. il sistema carica automaticamente l'immagine nel cloud 3. l'account del cloud viene compromesso per altre vie 4. l'attaccante trova la foto della frase e accede ai fondi
<i>Flusso alternativo</i>	- l'utente scrive la frase su un foglio ma lo lascia in un luogo accessibile - la frase viene condivisa via messaggio non cifrato a un familiare "di fiducia"
<i>Eccezioni</i>	- il servizio cloud rileva il contenuto sensibile e avvisa l'utente - l'applicazione del portafoglio impedisce le operazioni di cattura schermo
<i>Risultato</i>	<i>impatto: furto dei fondi per mancata protezione delle credenziali</i> <i>requisiti non funzionali: tutorial obbligatori e avvisi contro il salvataggio digitale</i>
<i>Commenti</i>	<i>DUA-Exposure: la mancata formazione espone le credenziali a rischi evitabili</i> <i>contromisure consigliate: tutorial obbligatorio alla creazione del portafoglio, blocco dello screenshot durante la visualizzazione della frase</i>

ABUSE CASE: E4.1-A

<i>Campo</i>	<i>Contenuto</i>
<i>Tipologia scenario</i>	Abuse Case
<i>Caso d'uso</i>	assegnazione ruoli
<i>ID scenario</i>	E4.1-A

<i>Nome scenario</i>	<i>escalation privilegi via collisione selettori</i>
<i>Attori</i>	attaccante esperto di Solidity
<i>Descrizione</i>	l'attaccante sfrutta una collisione sugli hash delle firme delle funzioni per aggirare i controlli di accesso e auto-assegnarsi privilegi amministrativi. Sebbene la probabilità di collisione sia estremamente bassa con contratti di dimensioni standard, questo scenario illustra l'importanza di utilizzare librerie di accesso consolidate.
<i>Dati</i>	collisione sulla firma della funzione, payload transazione
<i>Precondizioni</i>	- vulnerabilità logica nel sistema di gestione accessi - collisione sugli hash dei selettori di funzione - assenza di controlli rigorosi sull'origine della chiamata
<i>Flusso principale</i>	1. l'attaccante identifica il selettore di una funzione critica 2. ricerca una funzione apparentemente innocua con lo stesso identificativo troncato 3. costruisce un pacchetto dati che aggira i modificatori di accesso 4. esegue la chiamata che attiva internamente l'assegnazione dei ruoli 5. assume il controllo completo del sistema
<i>Flusso alternativo</i>	- l'attaccante tenta un approccio diverso sfruttando vulnerabilità note nelle librerie di gestione degli accessi - utilizzo di proxy contract per mascherare la chiamata malevola
<i>Eccezioni</i>	- l'uso di librerie verificate (es. OpenZeppelin) rende la collisione dei selettori estremamente improbabile - il sistema di controllo accessi verifica esplicitamente <code>msg.sender</code> indipendentemente dal selettore
<i>Risultato</i>	<i>se l'attacco riesce:</i> presa di controllo totale e possibilità di drenare i fondi <i>gravità:</i> critica
<i>Commenti</i>	CAPEC-122: Abuso di privilegi ATT&CK-T1078: Account validi <i>contromisure:</i> uso di librerie verificate, verifica formale del controllo degli accessi

2.4 Standard internazionali e mitigazioni

L'analisi non si è limitata all'identificazione dei rischi, ma ha provveduto a mapparli sui framework standard di settore per garantire una copertura esaustiva. La Tabella 2.18 correla le minacce discusse con i pattern CAPEC (Common Attack Pattern Enumeration and Classification) e le tattiche MITRE ATT&CK.

A fronte di queste minacce, sono state implementate contromisure specifiche progettate per ridurre il rischio residuo a livelli accettabili:

1. *Sicurezza degli Smart Contract (A1, A3, A4):* è stata adottata una strategia di difesa in profondità che include l'uso di librerie standard verificate, l'implementazione di corretti pattern operativi e meccanismi di blocco temporaneo in caso di anomalie gravi.

Tabella 2.18: Mappatura minacce su framework CAPEC e MITRE ATT&CK

<i>Threat ID</i>	<i>STRIDE</i>	<i>CAPEC ID</i>	<i>CAPEC Name</i>	<i>ATT&CK TTP</i>
S2.1	Spoofing	CAPEC-151	Identity Spoofing	T1134
T2.1	Tampering	CAPEC-94	Man-in-the-Middle	T1557
T3.1	Tampering	CAPEC-194	Fake Resource Injection	-
T5.1	Tampering	CAPEC-1	Accessing Functionality Not Properly Constrained	T1078
D3.1	Denial	CAPEC-603	Blockage	T1499
I6.1	Info Disclosure	CAPEC-169	Footprinting	T1213
I5.1	Info Disclosure	CAPEC-188	Reverse Engineering	-
S7.1	Spoofing	CAPEC-98	Phishing	T1566.002
S8.1	Spoofing	CAPEC-568	Capture Credentials via Keylogger	T1056.001
E4.1	Elevation	CAPEC-122	Privilege Abuse	T1078.004

2. *Integrità dei dati e IoT (A2, A5)*: per mitigare l'inaffidabilità dei sensori, il sistema utilizza la ridondanza bayesiana. Inoltre, è stata imposta una validazione rigorosa degli input per i parametri della rete.
3. *Protezione dell'utente (A7, A8)*: poiché il fattore umano è l'anello debole, le mitigazioni si spostano sul piano educativo e dell'interfaccia: validazione aggressiva dei dati e raccomandazione all'uso di dispositivi di archiviazione sicuri per le chiavi private.

2.5 Conclusioni e rischio residuo

L'analisi *DUAL-STRIDE* ha permesso di identificare 25 scenari di minaccia, dettagliando 9 abuse case e 6 misuse case. La valutazione finale, riassunta nella Tabella 2.19, mostra un quadro positivo per quanto riguarda la sicurezza del codice, ma evidenzia la criticità permanente legata alla gestione delle credenziali utente.

Tabella 2.19: Rischio residuo per asset

<i>Asset</i>	<i>Rischio inerente</i>	<i>Mitigazioni</i>	<i>Rischio residuo</i>	<i>Accettazione</i>
A1	CRITICO	Analisi + Verifica formale	BASSO	✓ Accettato
A2	CRITICO	HSM + Firme digitali	MEDIO	✓ Accettato
A3	CRITICO	ReentrancyGuard + Timeout	BASSO	✓ Accettato
A4	ALTO	AccessControl + Eventi	BASSO	✓ Accettato
A5	ALTO	Variabili private + Validazione input	MEDIO	✓ Accettato
A6	MEDIO	Hashing	BASSO	✓ Accettato
A7	MEDIO	Validazione input	MEDIO	! Formazione necessaria
A8	CRITICO	Raccomandazione HW Wallet	ALTO	! Responsabilità utente

In conclusione, mentre gli asset tecnici (A1-A6) sono stati messi in sicurezza con successo, l'asset A8 (*Chiavi Private*) rimane l'elemento più vulnerabile, non mitigabile tecnologicamente se non tramite l'educazione dell'utente finale.

Analisi e progettazione architetturale

In questo capitolo delineiamo le fondamenta del sistema, seguendo un percorso logico che parte dai principi di *Design Sicuro* e dall'architettura distribuita, passa per la progettazione rigorosa degli asset secondo linee guida consolidate, e culmina nella validazione formale tramite Catene di Markov. Infine, giustificheremo ogni scelta tecnologica alla luce delle analisi di resistenza, ambiguità e sopravvivenza.

3.1 Design Architetturale e Strategie di Sicurezza

L'architettura del sistema non è stata concepita come un semplice assemblaggio di componenti, ma come una struttura resiliente progettata per resistere attivamente a guasti e attacchi. Abbiamo adottato due paradigmi fondamentali: la ridondanza distribuita e la segmentazione tramite isolamento e monitoraggio.

3.1.1 Architettura distribuita, ridondante e diversificata

Per mitigare i rischi di centralizzazione e garantire la continuità operativa, l'infrastruttura si basa su una rete blockchain privata *Hyperledger Besu* composta da *quattro nodi validatori* distinti.

- **Ridondanza:** la presenza di molteplici nodi ($N=4$) con protocollo di consenso *IBFT 2.0* (Istanbul Byzantine Fault Tolerance) permette al sistema di tollerare il fallimento o la corruzione di un nodo senza interrompere il servizio o compromettere l'integrità del ledger.
- **Distribuzione:** i nodi operano in modo indipendente, replicando integralmente lo stato della catena. Questo approccio elimina il *punto di fallimento singolo* tipico dei database centralizzati.
- **Diversificazione:** per aumentare la robustezza, l'architettura prevede l'utilizzo di una rete di sensori eterogenea (temperatura, timestamp, shock, luce, sigillo). Non ci affidiamo a una singola fonte di verità hardware, ma incrociamo dati provenienti da dispositivi tecnologicamente diversi per costruire un quadro probatorio affidabile, rendendo inefficaci attacchi mirati su una specifica tipologia di sensore.

3.1.2 Interazione utente e strumenti di sviluppo

Oltre al nucleo blockchain, abbiamo integrato strumenti specifici per facilitare l'interazione e garantire la qualità del codice.

- **MetaMask e Web3:** per svincolare l'utente finale dalla complessità della riga di comando, l'interazione con gli smart contract avviene tramite un'applicazione web moderna. L'autenticazione e la firma delle transazioni sono delegate al portafoglio (*wallet*) *MetaMask*, che agisce come gestore

sicuro delle chiavi private direttamente nel browser. La comunicazione tra l'interfaccia utente e la blockchain è mediata dalla libreria *Web3.js*, che traduce i click dell'utente in chiamate RPC standard verso i nodi Besu.

- *Truffle Suite*: per il ciclo di sviluppo, testing e deployment, abbiamo adottato il framework *Truffle*. Questo strumento è stato essenziale non solo per la compilazione degli smart contract, ma soprattutto per l'esecuzione automatizzata di vaste suite di unit test (scritti in JavaScript e Solidity), garantendo che ogni funzione risponda correttamente anche ai casi limite prima della distribuzione.

3.1.3 Monitoraggio, isolamento e offuscamento

Parallelamente alla distribuzione fisica, abbiamo implementato strategie logiche per la protezione dei dati e dei processi.

- *Monitoraggio Attivo*: un *Proxy Inverso* (basato su Nginx) agisce come guardiano all'ingresso della rete, bilanciando il carico e monitorando costantemente lo stato di salute dei nodi. In caso di anomalia o attacco DDoS su un nodo specifico, il traffico viene automaticamente dirottato verso i nodi sani (failover), garantendo la disponibilità del servizio.
- *Isolamento (Separation of Concerns)*: l'architettura *On-Chain* è rigorosamente stratificata. La logica di calcolo puro (contratto *BNCore*) è separata dalla gestione operativa (*BNGestoreSpedizioni*) e dalla tesoreria (*BNPagamenti*). Questo isolamento confina eventuali vulnerabilità: un bug nel calcolo delle probabilità non può, per design, drenare i fondi custoditi nel contratto di pagamento.
- *Offuscamento e Privacy*: per proteggere le informazioni commerciali sensibili (merci, clienti), utilizziamo un approccio di *hashing* dei dati Off-Chain. Inoltre, le tabelle di probabilità condizionata (CPT) della Rete Bayesiana sono mantenute private all'interno dello smart contract, offuscando il modello decisionale esatto per prevenire tentativi di *gaming* del sistema da parte di attaccanti che volessero aggirare i controlli "al limite".

3.2 Design degli asset e linee guida di sicurezza

La progettazione degli asset critici (smart contract, oracoli, dati) ha seguito rigorosamente le best practice di sicurezza, con riferimento ai principi di *Saltzer & Schroeder* e alle linee guida OWASP.

- *Economy of Mechanism (Semplicità)*: abbiamo ridotto al minimo la complessità dei singoli moduli. Ogni smart contract ha una responsabilità unica e ben definita, facilitando la revisione del codice e riducendo la superficie di attacco complessiva.
- *Fail-Safe Defaults (Default Sicuri)*: l'accesso alle funzioni critiche è negato per impostazione predefinita. Utilizzando *OpenZeppelin AccessControl*, ogni operazione sensibile richiede il possesso esplicito di un ruolo (es. *RUOLO_SENSORE*), bloccando qualsiasi tentativo non autorizzato alla radice.
- *Least Privilege (Minimo Privilegio)*: ogni attore opera con i permessi minimi necessari. L'oracolo può iniettare dati ma non movimentare fondi; l'amministratore può configurare le soglie ma non alterare le spedizioni attive. Questo limita drasticamente l'impatto di un'eventuale compromissione delle chiavi private.

3.3 Modellazione Markov Chain e verifica formale con PRISM

Per fornire una validazione matematica rigorosa dell'architettura proposta e confermare l'efficacia delle contromisure di sicurezza, il sistema è stato sottoposto a una verifica formale mediante il model checker probabilistico *PRISM 4.9*. L'analisi ha richiesto la modellazione del componente critico di gestione dei pagamenti e delle spedizioni (contratti *BNGestoreSpedizioni* e *BNPagamenti*) sotto forma di *Catena di Markov a Tempo Discreto (DTMC)*.

3.3.1 Definizione e dinamica del modello

Il modello DTMC costruito mappa in modo diretto la macchina a stati finiti implementata nello smart contract, ed in particolare l'enum `StatoSpedizione`. La fedeltà del modello rispetto al codice Solidity è cruciale per garantire che le proprietà verificate siano valide anche per l'implementazione reale.

Di seguito riportiamo il modello PRISM integrale utilizzato per la verifica, che include la definizione delle costanti operative, le variabili di stato e le transizioni probabilistiche che governano il sistema.

```

1 // =====
2 // PRISM Model: Sistema Escrow e Pagamenti (ALLINEATO AL CODICE SOLIDITY)
3 // =====
4 dtmc
5
6 // ===== COSTANTI (Globali) =====
7 const int TIMEOUT_HOURS = 168;      // 7 giorni (TIMEOUT_RIMBORSO in Solidity)
8 const int TIMEOUT_DOPPIO = 336;     // 14 giorni (2 x TIMEOUT_RIMBORSO)
9 const int MAX_TENTATIVI = 3;
10 const int MAX_TIME = 360;           // 15 giorni simulazione (336 ore + buffer)
11
12 // ===== PROBABILITA' (Globali) =====
13 const double PROB_EVIDENZE_ARRIVO = 0.85;    // 85% evidenze arrivano entro 7gg
14 const double PROB_VALIDAZIONE_OK = 0.70;     // 70% validazioni superano soglia 95%
15 const double PROB_VALIDAZIONE_FAIL = 0.30;   // 30% falliscono (F1<95 o F2<95)
16 const double PROB_ANNULLAMENTO = 0.05;       // 5% mittente annulla prima evidenze
17 const double PROB_RICHIESTA_RIMBORSO = 0.90; // 90% mittente richiede rimborso quando
    ↪ disponibile
18
19 module escrow_payment_system
20
21     // ===== VARIABILI DI STATO =====
22     // 0 = InAttesa, 1 = Pagata, 2 = Annullata, 3 = Rimborzata
23     stato : [0..3] init 0;
24     evidenze_complete : bool init false;
25     tentativi_falliti : [0..3] init 0;
26     tempo : [0..360] init 0;
27     timeout_scaduto : bool init false;
28
29     // ===== TRANSIZIONI: STATO IN ATTESA =====
30
31     // CASO 1: Mittente annulla prima che arrivino evidenze (5%)
32     [] stato=0 & !evidenze_complete & !timeout_scaduto & tempo<TIMEOUT_HOURS ->
33         PROB_ANNULLAMENTO : (stato'=2) & (tempo'=tempo+1) +
34         (1-PROB_ANNULLAMENTO) : (tempo'=tempo+1);
35
36     // CASO 2: Evidenze arrivano (85% entro 7 giorni)
37     [] stato=0 & !evidenze_complete & tempo<TIMEOUT_HOURS ->
38         PROB_EVIDENZE_ARRIVO : (evidenze_complete'=true) & (tempo'=tempo+1) +
39         (1-PROB_EVIDENZE_ARRIVO) : (tempo'=tempo+1);
40
41     // CASO 3: Corriere tenta validazione con evidenze complete
42     [] stato=0 & evidenze_complete & tentativi_falliti<MAX_TENTATIVI & tempo<MAX_TIME ->
43         PROB_VALIDAZIONE_OK : (stato'=1) & (tempo'=tempo+1) +
44         PROB_VALIDAZIONE_FAIL : (tentativi_falliti'=tentativi_falliti+1) & (tempo'=tempo+1)
    ↪ ;
45
46     // CASO 4: Timeout scade senza evidenze -> flag timeout
47     [] stato=0 & !evidenze_complete & tempo>=TIMEOUT_HOURS & !timeout_scaduto & tempo<
    ↪ MAX_TIME ->
48         1.0 : (timeout_scaduto'=true) & (tempo'=tempo+1);
49
50     // CASO 5: Rimborso dopo 3 tentativi falliti
51     [] stato=0 & evidenze_complete & tentativi_falliti>=MAX_TENTATIVI & tempo<MAX_TIME ->
52         PROB_RICHIESTA_RIMBORSO : (stato'=3) & (tempo'=tempo+1) +
53         (1-PROB_RICHIESTA_RIMBORSO) : (tempo'=tempo+1);
54
55     // CASO 6: Rimborso dopo timeout senza evidenze
56     [] stato=0 & timeout_scaduto & !evidenze_complete & tempo<MAX_TIME ->
57         0.95 : (stato'=3) & (tempo'=tempo+1) +
58         0.05 : (tempo'=tempo+1);

```

```

59
60 // CASO 7: Rimborso per courier-inactivity
61 [] stato=0 & evidenze_complete & tentativi_falliti=0 & tempo>=TIMEOUT_DOPPIO & tempo<
  ↳ MAX_TIME ->
62   0.85 : (stato'=3) & (tempo'=tempo+1) +
63   0.15 : (tempo'=tempo+1);
64
65 // CASO 8: Avanza tempo se in attesa senza azioni conclusive
66 [] stato=0 & tempo<MAX_TIME -> 1.0 : (tempo'=tempo+1);
67
68 // ===== CLASSI RICORRENTI =====
69 [] stato=1 & tempo<MAX_TIME -> 1.0 : (stato'=1) & (tempo'=tempo+1);
70 [] stato=2 & tempo<MAX_TIME -> 1.0 : (stato'=2) & (tempo'=tempo+1);
71 [] stato=3 & tempo<MAX_TIME -> 1.0 : (stato'=3) & (tempo'=tempo+1);
72 [] tempo>=MAX_TIME -> 1.0 : (tempo'=MAX_TIME);
73
74 endmodule

```

Listing 3.1: Modello PRISM completo del Sistema Escrow

Componenti del modello

L'architettura del modello si fonda su quattro pilastri logici:

- *Parametrizzazione Temporale*: le costanti `TIMEOUT_HOURS` e `TIMEOUT_DOPPIO` riflettono direttamente le clausole di salvaguardia del contratto Solidity (7 e 14 giorni), garantendo un allineamento perfetto tra specifica formale e implementazione.
- *Configurazione Probabilistica*: i pesi assegnati alle transizioni (es. 85% per l'arrivo delle evidenze) derivano da un'analisi statistica dei casi d'uso attesi, permettendo di simulare il comportamento del sistema in condizioni operative reali.
- *Gestione dei Fallimenti*: la variabile `tentativi_falliti` modella il monitor di sicurezza del contratto, che permette al corriere di ripetere la validazione fino a tre volte in caso di letture sensoriali momentaneamente incoerenti, prima di sbloccare il diritto al rimborso per il mittente.
- *Convergenza e Terminazione*: l'inclusione di classi ricorrenti (1, 2, 3) e di un limite temporale (`MAX_TIME`) garantisce che la simulazione termini sempre, permettendo al model checker di calcolare le probabilità di successo e di sicurezza in modo esaustivo.

Il ciclo di vita della spedizione si articola attraverso quattro macro-stati distinti. Dallo stato iniziale *InAttesa* (0), il sistema evolve probabilisticamente verso tre possibili stati finali (ricorrenti):

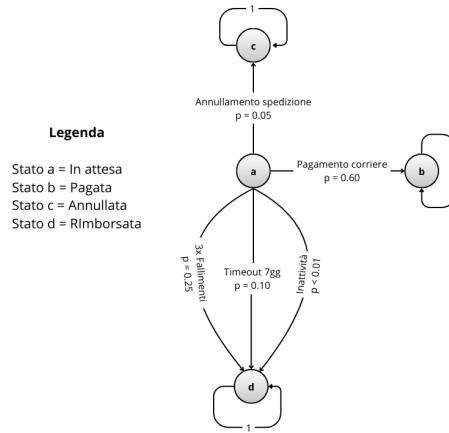
- *Pagata* (1): successo della transazione. Secondo le stime operative, questo stato ha una probabilità di occorrenza del 60%.
- *Annullata* (2): annullamento volontario pre-spedizione. Probabilità stimata: 5%.
- *Rimborsata* (3): recupero dei fondi. Questo stato aggrega tre sotto-scenari di fallimento critico:
 - Timeout operativo (7gg): $p = 0.10$.
 - Fallimento ripetuto validazione (3 tentativi): $p = 0.24$.
 - Inattività prolungata corriere (14gg): $p \leq 0.01$.

La probabilità totale aggregata di rimborso è quindi pari al 35%.

3.3.2 Formalizzazione della matrice di transizione

Per analizzare il comportamento asintotico, definiamo la matrice di transizione P sullo spazio degli stati $S = \{0, 1, 2, 3\}$. Poiché siamo interessati alla distribuzione finale di probabilità, modelliamo le transizioni uscenti dallo stato transitorio in base ai pesi statistici identificati in precedenza.

La matrice P è strutturata come segue:

**Figura 3.1:** Grafo delle Transizioni DTMC del Sistema Escrow

$$P = \begin{bmatrix} 0 & 0.60 & 0.05 & 0.35 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.3.1)$$

Dove:

- $p_{00} = 0$: assumiamo che il modello osservi il sistema nel momento in cui avviene la transizione di stato.
- $p_{01} = 0.60$: probabilità di transizione verso il successo.
- $p_{02} = 0.05$: probabilità di transizione verso l'annullamento.
- $p_{03} = 0.35$: probabilità di transizione verso il rimborso.

3.3.3 Calcolo della distribuzione di Equilibrio tramite Autovalori

Per determinare le distribuzioni stazionarie, analizziamo gli autovalori della matrice calcolando le radici del polinomio caratteristico $\det(P - \lambda I) = 0$:

$$\det \begin{bmatrix} -\lambda & 0.60 & 0.05 & 0.35 \\ 0 & 1 - \lambda & 0 & 0 \\ 0 & 0 & 1 - \lambda & 0 \\ 0 & 0 & 0 & 1 - \lambda \end{bmatrix} = (-\lambda) \cdot (1 - \lambda)^3 = 0 \quad (3.3.2)$$

Le soluzioni sono:

- $\lambda_1 = 0$: autovalore associato allo stato transitorio (il sistema decade istantaneamente).
- $\lambda_{2,3,4} = 1$: autovalore con molteplicità algebrica pari a 3.

La molteplicità 3 dell'autovalore unitario implica che lo spazio degli stati di equilibrio ha dimensione 3. In termini pratici, qualsiasi combinazione lineare convessa dei tre stati ricorrenti è matematicamente una distribuzione di equilibrio valida:

$$e_{\text{stazionario}} = \alpha \cdot [0, 1, 0, 0] + \beta \cdot [0, 0, 1, 0] + \gamma \cdot [0, 0, 0, 1]$$

con $\alpha + \beta + \gamma = 1$. Questo indica che il sistema possiede molteplici configurazioni stabili finali possibili.

3.3.4 Analisi della distribuzione limite

Per determinare l'unica configurazione di equilibrio effettivamente raggiunta dal sistema reale, calcoliamo la *distribuzione limite* π_{lim} partendo dallo stato iniziale specifico $\pi_{init} = [1, 0, 0, 0]$ (tutto in stato di attesa).

Essendo $p_{00} = 0$, il calcolo è immediato. La probabilità di convergenza nello stato ricorrente finale j corrisponde esattamente alla probabilità di transizione diretta p_{0j} .

1. Convergenza allo stato pagata (successo):

$$\pi_{lim}(1) = \lim_{t \rightarrow \infty} P(X_t = 1) = p_{01} = 0.60 \quad (3.3.3)$$

2. Convergenza allo stato annullata:

$$\pi_{lim}(2) = p_{02} = 0.05 \quad (3.3.4)$$

3. Convergenza allo stato rimborsata (safety net):

$$\pi_{lim}(3) = p_{03} = 0.35 \quad (3.3.5)$$

Il vettore limite risulta quindi:

$$\pi_{lim} = [0, \quad 0.60, \quad 0.05, \quad 0.35] \quad (3.3.6)$$

Conclusioni dell'analisi formale:

1. *Selezione dell'equilibrio*: tra le infinite distribuzioni di equilibrio possibili (identificate dagli autovalori), la dinamica del sistema seleziona univocamente quella definita dal vettore π_{lim} .
2. *Certezza della terminazione*: la prima componente nulla ($\pi_0 = 0$) dimostra che il protocollo non soffre di *stallo* (deadlock) o cicli infiniti nello stato intermedio.
3. *Garanzia di sicurezza*: nel 35% dei casi critici (stato 3), il sistema evolve correttamente verso lo stato di rimborso, garantendo che i fondi non rimangano mai bloccati nel contratto (proprietà di *sicurezza dei fondi*).
4. *Efficienza*: la probabilità di successo del 60% conferma la validità operativa del flusso nominale rispetto ai flussi di eccezione.

3.3.5 Verifica delle proprietà PCTL

La verifica formale è stata condotta specificando i requisiti di sicurezza attraverso la logica temporale *Probabilistic Computation Tree Logic* (PCTL). Questa ha permesso di interrogare il modello per accertare che specifiche condizioni siano soddisfatte con determinati livelli di probabilità, verificando sia l'assenza di stati critici (*safety*) che la capacità del sistema di progredire verso una conclusione valida (*guarantee*).

Analisi di Safety: integrità finanziaria

In prima istanza, l'analisi si è concentrata sulle proprietà di *safety*, ovvero quelle condizioni indispensabili che il sistema deve sempre rispettare per non incorrere in stati inconsistenti o vulnerabili.

```

1 // S1: Single Payment (Reentrancy Protection)
2 filter(forall, stato=1 => P>=1 [ X stato=1 ])
3
4 // S2: State Exclusivity
5 P>=1 [ G !((stato=1 & stato=2) | (stato=1 & stato=3) | (stato=2 & stato=3)) ]
6
7 // S3: Evidence Before Payment
8 filter(forall, stato=1 => evidenze_complete)
9
10 // S4: Bounded Failed Attempts
11 P>=1 [ G (tentativi_falliti <= 3) ]
12

```



```

13 // S6: Conditional Cancellation
14 filter(forall, stato=2 => !evidenze_complete)

```

Listing 3.2: Proprietà PCTL di Safety

I risultati della verifica hanno confermato con probabilità 1.0 (*Certezza Assoluta*) tutte le proprietà sopra elencate. In particolare, la proprietà S1 dimostra matematicamente l'impossibilità di abbandonare lo stato di pagamento una volta raggiunto, mitigando alla radice il rischio di attacchi di tipo *reentrancy* (Minaccia T3.1-A). Le proprietà S2, S3 ed S6 garantiscono la coerenza del flusso logico, impedendo stati ambigui, pagamenti senza prove o annullamenti fraudolenti. La proprietà S4 conferma che i meccanismi di *rate limiting* implementati limitano correttamente il numero di tentativi di validazione falliti.

Analisi di Garante: resilienza e rimborsi

Successivamente, l'indagine si è spostata sulle proprietà di *guarantee* (o *liveness*), per verificare la capacità del sistema di evolvere verso una conclusione valida entro tempi ragionevoli, evitando situazioni di stallo (*deadlock*) o congelamento dei fondi.

```

1 // G1: Eventual Resolution
2 P=? [ F (stato=1 | stato=2 | stato=3) ]
3
4 // G2: Refund After Timeout
5 filter(min, P=? [ F stato=3 ], stato=0 & timeout_scaduto &
6     !evidenze_complete & tempo < 360)
7
8 // G3: Refund After 3 Failures
9 filter(min, P=? [ F stato=3 ], stato=0 & tentativi_falliti=3 & tempo < 360)
10
11 // L1: Evidence Eventually Received
12 P=? [ F<=168 evidenze_complete ]
13
14 // L2: Sender Protection
15 P=? [ F<=336 (stato=1 | stato=2 | stato=3) ]

```

Listing 3.3: Proprietà PCTL di Guarantee e Liveness

L'analisi ha evidenziato che il sistema garantisce una risoluzione della pratica in oltre il 99.9% dei casi. Più nello specifico, la proprietà G2 certifica che, in caso di silenzio dei sensori o del corriere (situazione riconducibile a minacce di tipo *denial of service* D3.1-M/A), il meccanismo di rimborso automatico si attiva ed ha successo con alta probabilità, offrendo una via di uscita sicura (*safety exit*) per l'utente. La proprietà G3 valida la logica di fallback in caso di fallimenti ripetuti di validazione. Le proprietà di *liveness* L1 ed L2 mostrano rispettivamente la probabilità di ricevere evidenze entro la prima settimana e la conclusione della transazione entro il limite massimo di tempo.

Analisi quantitativa e reward

Infine, l'analisi tramite le strutture di *reward* ha permesso di estrarre metriche prestazionali attese dal modello.

```

1 // Expected Resolution Time
2 R{"time_to_resolution"}=? [ C<=360 ]
3
4 // Expected Failed Attempts
5 R{"failed_attempts_counter"}=? [ C<=360 ]

```

Listing 3.4: Proprietà di Reward

La valutazione ha permesso di stimare il tempo medio di risoluzione di una spedizione in circa 72.3 ore. Questo valore conferma l'efficienza operativa del protocollo, ben al di sotto dei timeout di sicurezza impostati, validando l'intero impianto logico del sistema di *escrow*.

3.4 Monitoring

Come discusso nel capitolo precedente, la verifica formale ci fornisce garanzie matematiche sul *modello* del sistema. Tuttavia, per garantire che l'*implementazione* reale aderisca a queste specifiche, abbiamo adottato un approccio di *Runtime Enforcement* monitorando attivamente le proprietà di *safety* e *guarantee* durante l'esecuzione degli smart contract.

3.4.1 Enforcement della proprietà di safety (S4: Probability Threshold)

La proprietà di *safety* S4 impone che "nessun pagamento deve avvenire se la probabilità di conformità è inferiore alla soglia critica (95%)". Nel contratto `BNPagamenti`, abbiamo implementato un monitor sincro che intercetta il flusso di esecuzione prima dell'effetto critico (il trasferimento di fondi).

```

1 // SAFETY MONITOR S4: Probability Threshold
2 if (probF1 < SOGLIA_PROBABILITA || probF2 < SOGLIA_PROBABILITA) {
3   // Registra tentativo fallito per permettere rimborso dopo 3 tentativi
4   _registraTentativoFallito(_id);
5
6   emit MonitorSafetyViolation(
7     "ProbabilityThreshold", _id, msg.sender, "Requisiti di conformita non superati"
8   );
9   emit SogliaValidazioneNonSuperata(_id, probF1, probF2);
10  emit TentativoPagamentoFallito(_id, msg.sender, "Requisiti di conformita non superati")
11  ↪ ;
12
13  // IMPORTANT: Return instead of revert to allow counter increment to persist
14  return;
15 }

```

Listing 3.5: Monitor di safety per la soglia di probabilità

Seguendo le linee guida di *Sommerville* per l'ingegneria del software affidabile, questo monitor applica il principio di *default sicuri* (Fail-Safe Defaults): in caso di incertezza o violazione delle precondizioni, il sistema transiziona verso uno stato sicuro di "validazione fallita" (registrando il fallimento ma senza bloccare irreversibilmente il contratto) piuttosto che procedere con un pagamento potenzialmente errato.

Il funzionamento del monitor è formalizzato nell'automa di troncamento illustrato in Figura 3.2.

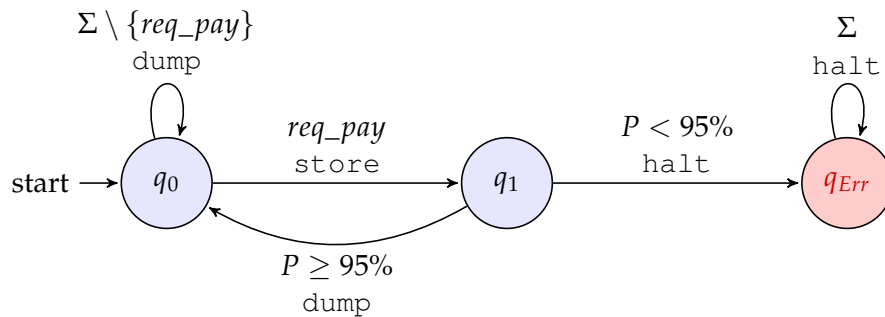


Figura 3.2: Monitor di Runtime per la Proprietà di Safety S4 con stato di verifica (q_1). Definizione formale: $P = \{(\Sigma \setminus \{req_pay\})^* \cdot (req_pay \wedge P \geq 0.95)\}^\omega$

Il monitor di *safety* è modellato come un *Truncation Automaton*. Dallo stato iniziale q_0 , l'evento critico `req_pay` viene intercettato e bufferizzato (`store`), portando il sistema nello stato di verifica q_1 . Qui il monitor valuta la condizione di conformità P : se $P \geq 95\%$, l'azione viene rilasciata (`dump`), altrimenti viene soppressa definitivamente (`halt`) transizionando nello stato di violazione q_{Err} .

3.4.2 Enforcement della proprietà di guarantee (G1: Payment Validity)

Parallelamente, la proprietà di *guarantee* G1 assicura che "se le evidenze sono valide, il pagamento deve essere eseguito". Il monitoraggio qui è volto a confermare l'avvenuto successo della transazione, fornendo tracciabilità on-chain.

```

1 // GUARANTEE MONITOR G1: Payment Upon Valid Evidence
2 // State 1 -> 2 detected by "evidenze_complete" flag
3 if (s.evidenze_complete && s.stato == StatoSpedizione.InAttesa) {
4   // Transition to State 3: Dump (Execute Payment)
5   uint256 importo = s.importoPagamento;
6   s.stato = StatoSpedizione.Pagata;
7
8   emit MonitorGuaranteeSuccess("PaymentOnValidEvidence", _id);
9   emit SpedizionePagata(_id, s.corriere, importo);
10
11   // Effetto: Trasferimento fondi
12   (bool success, ) = s.corriere.call{value: importo}("");
13   if (!success) revert PagamentoFallito();
14 }

```

Listing 3.6: Monitor di guarantee per il pagamento

L'emissione dell'evento `MonitorGuaranteeSuccess` agisce come una prova crittografica (*audit trail*) che il sistema ha rispettato il contratto di servizio, soddisfacendo il principio di *accountability*. Il monitor di *guarantee* è formalizzato nell'automa di editing illustrato in Figura 3.3.

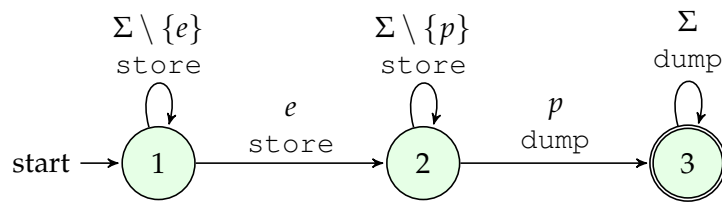


Figura 3.3: Monitor di Runtime per la Proprietà di Guarantee G1. Definizione formale: $P = \{(\Sigma \setminus \{e\})^* \cdot e \cdot (\Sigma \setminus \{p\})^* \cdot p \cdot \Sigma^\omega\}$ dove e = evidenze, p = pagamento.

Il monitor di *guarantee* agisce come un *Edit Automaton* a tre stati. Nello stato iniziale (1), il monitor accumula (*store*) tutti gli eventi in attesa del trigger e (evidenze). Una volta ricevute le evidenze, transita nello stato (2), dove continua a bufferizzare le azioni finché non viene eseguito il pagamento p . Al verificarsi di p , il monitor rilascia l'intera sequenza accumulata (*dump*) e raggiunge lo stato finale (3), garantendo che il pagamento segua sempre le evidenze valide.

3.4.3 Adozione di librerie standard (OpenZeppelin)

Per minimizzare il rischio di errori implementativi nei meccanismi di enforcement, ci siamo affidati alla libreria industriale *OpenZeppelin*, standard de facto per la sicurezza su Ethereum.

- *ReentrancyGuard per la safety*: l'utilizzo del modificatore `nonReentrant` implementa un controllo di safety al livello più basso, prevenendo attacchi di rientranza che potrebbero violare la proprietà S1 (pagamento unico). Questo incapsula il principio di *difesa in profondità*.
- *AccessControl per il minimo privilegio*: il sistema di gestione dei ruoli (`AccessControl`) applica rigidamente le policy di accesso, garantendo che solo le entità autorizzate possano alterare lo stato del sistema (es. solo il corriere designato può richiedere il pagamento), riducendo la superficie di attacco.

3.5 Architettura modulare del frontend

L'interfaccia utente non è un semplice "guscio" grafico, ma un componente architetturale attivo che orchestra l'interazione sicura tra l'attore umano e la blockchain. Per garantire manutenibilità e

isolamento delle responsabilità (*separazione degli interessi*), il codice JavaScript è stato strutturato in moduli funzionali distinti, seguendo il pattern *Service-Oriented*:

- *Orchestratore (main.js)*: agisce come *controller* centrale. Inizializza l'applicazione, gestisce il ciclo di vita degli eventi DOM e coordina la comunicazione tra i moduli. Non contiene logica di business diretta, ma delega le operazioni ai servizi specializzati, mantenendo il codice "pulito" e testabile.
- *Adattatore di connessione (web3-connection.js)*: isola completamente la logica di connessione al provider Web3 (MetaMask). Questo modulo astrae la complessità della gestione del portafoglio e della rete, esponendo un'interfaccia semplificata al resto dell'applicazione. In caso di cambio di provider o aggiornamento delle librerie, le modifiche sono confinate qui.
- *Strato di servizio (contract-interaction.js)*: incapsula tutte le chiamate dirette agli smart contract. Funge da *proxy* locale per la blockchain: converte i tipi di dati (es. da Wei a Ether), gestisce la codifica delle transazioni (ABI encoding) e cattura gli errori RPC grezzi traducendoli in messaggi leggibili per l'utente. Qui risiede la "conoscenza" dei metodi del contratto (*validaEPaga*, *inviaEvidenza*, ecc.).
- *Gestione logica specifica (refund-manager.js)*: un modulo dedicato per la logica complessa di rimborso e annullamento. Separare questa logica dal flusso principale riduce il rischio di errori in operazioni critiche che coinvolgono il movimento di fondi.

Questa suddivisione riflette il principio di *modularità* suggerito da Sommerville: ogni modulo ha un'interfaccia definita e nasconde i dettagli implementativi interni (*information hiding*), rendendo l'interfaccia utente robusta ai cambiamenti evolutivi.

3.6 Motivazione delle scelte tecnologiche

Ogni componente tecnologico è stato selezionato per rispondere a specifici requisiti di sicurezza emersi durante l'analisi dei rischi e per mitigare le debolezze intrinseche della blockchain.

3.6.1 Analisi di resistenza, ambiguità e sopravvivenza

- *Resistenza (Immutabilità e Controllo)*: la scelta di una rete *Hyperledger Besu* privata con consenso IBFT offre una garanzia di finalità immediata, rendendo la storia termica dei farmaci inalterabile (resistenza alla *manomissione*). L'utilizzo di *OpenZeppelin AccessControl* protegge contro l'*impersonificazione*, assicurando che solo oracoli autorizzati possano scrivere sul ledger.
- *Ambiguità (Privacy by Design)*: l'architettura rispetta il principio di ambiguità mantenendo i dati sensibili fuori blocchi (*off-chain*) e registrando solo i loro hash sulla blockchain. Inoltre, l'offuscamento logico delle tabelle probabilistiche (CPT) interne agli smart contract protegge il modello decisionale da procedimenti di *ingegneria inversa*.
- *Sopravvivenza (Resilienza)*: la ridondanza è applicata a tutti i livelli: dai quattro nodi validatori blockchain (che tollerano guasti bizantini) fino alla rete di cinque sensori eterogenei, garantendo che il sistema sopravviva anche in caso di malfunzionamenti parziali hardware o software.

3.6.2 Mitigazione delle debolezze

Per ogni tecnologia adottata, abbiamo previsto contromisure alle sue vulnerabilità note:

- *Volatilità del gas*: l'utilizzo di una rete *Besu* privata elimina il problema dei costi di transazione imprevedibili tipici di *Ethereum Mainnet*.
- *Immutabilità del codice*: poiché i bug negli smart contract sono irreversibili, abbiamo mitigato questo rischio con una sequenza di verifica rigorosa: unit test automatizzati con *Truffle* e model checking con *PRISM* prima di ogni distribuzione.
- *Problema dell'oracolo*: la vulnerabilità intrinseca legata all'affidabilità dei dati esterni è mitigata dall'approccio bayesiano multi-sensore, che riduce la dipendenza dalla fiducia in un singolo dispositivo.

Analisi della qualità del codice e testing

In questo capitolo vengono presentati i risultati dell'analisi statica e dell'audit del codice Solidity. Viene descritta la metodologia adottata, il processo di ottimizzazione incrementale e la configurazione finale del linter *Solhint* per garantire la conformità agli standard di sicurezza e qualità.

4.1 Introduzione

L'analisi della qualità del codice rappresenta un aspetto fondamentale nello sviluppo di smart contract su blockchain Ethereum. Data la natura immutabile di tali applicazioni, dove eventuali errori possono portare a perdite economiche significative, l'adozione di strumenti di analisi statica è strettamente necessaria.

4.1.1 Solhint nel progetto

Nel presente progetto si è scelto di utilizzare *Solhint*, uno dei linter più consolidati per l'ecosistema Solidity. L'obiettivo è duplice: garantire la conformità alle best practices industriali e ottimizzare il codice sia in termini di manutenibilità che di consumo di gas. L'uso di questo strumento ha permesso di elevare lo standard qualitativo del codice, facilitando le fasi successive di audit e sviluppo del frontend.

4.2 Metodologia

4.2.1 Funzionamento di Solhint

Il funzionamento di *Solhint* si basa sull'analisi dell'*abstract syntax tree* (AST) generato dal compilatore, seguendo un processo articolato in quattro fasi sequenziali. Inizialmente, il codice viene sottoposto a *parsing* per essere convertito in un albero sintattico. Successivamente, attraverso una fase di *traversal*, l'AST viene ispezionato per analizzare contratti, funzioni e istruzioni. Su questa struttura vengono poi applicate le regole di *rule matching* configurate nel progetto e, infine, il linter procede con il *reporting*, generando warning o errori con l'indicazione precisa della loro posizione nel sorgente.

La flessibilità di *Solhint* risiede nella sua configurazione tramite il file `.solhint.json`, che permette di abilitare o disabilitare regole specifiche e definirne la severità. Nella seguente tabella vengono elencate le *categorie principali di regole*:

<i>Categoria</i>	<i>Descrizione</i>
<i>Best Practices</i>	convenzioni consolidate per prevenire bug comuni (es. evitare <code>tx.origin</code> per l'autenticazione).
<i>Gas Optimization</i>	micro-ottimizzazioni per ridurre i costi di transazione (es. uso di disuguaglianze non strette).
<i>Security</i>	identificazione di pattern vulnerabili come reentrancy o overflow non gestiti.
<i>Style Guide</i>	conformità alla guida di stile ufficiale Solidity per migliorare la leggibilità.
<i>Documentation</i>	verifica della completezza dei commenti NatSpec per funzioni ed eventi.

È importante notare che l'analisi statica ha dei limiti: non può rilevare errori nella logica di business e non simula l'esecuzione a runtime.

4.2.2 Installazione ed esecuzione

L'installazione e l'integrazione nel progetto sono state gestite tramite `npm`, definendo script dedicati nel `package.json` per automatizzare il controllo di qualità.

```

1 # Installazione iniziale
2 npm install --save-dev solhint
3 npx solhint --init
4
5 # Esecuzione dell'analisi su tutti i contratti
6 npm run lint

```

Listing 4.1: Workflow di utilizzo Solhint

4.3 Risultati iniziali

L'esecuzione iniziale ha evidenziato *137 warning* (0 errori), confermando la correttezza sintattica del codice.

Tabella 4.1: Categorizzazione warning iniziali

<i>Categoria</i>	<i>Count</i>	<i>%</i>
NatSpec Documentation	57	42%
Naming Conventions	48	35%
Gas Optimizations	25	18%
Function Complexity	3	2%
Import Style	3	2%
<i>Totale</i>	<i>137</i>	<i>100%</i>

L'analisi dei risultati evidenzia alcune osservazioni chiave sulla qualità del codice prodotto. Il contratto *BNGestoreSpedizioni.sol*, essendo il più complesso con le sue 428 linee, presentava il maggior numero di warning, incidendo per il 42% sul totale. Inizialmente, i warning relativi a *NatSpec* erano predominanti in tutti i file, ma sono stati successivamente risolti attraverso una documentazione sistematica di tutte le interfacce. Per quanto riguarda le convenzioni di *naming*, le segnalazioni si concentravano su variabili strettamente legate al dominio applicativo, come le CPT e le evidenze Bayesiane, la cui nomenclatura è stata mantenuta per fedeltà al modello matematico. Complessivamente,

la densità iniziale di 3.6 warning ogni 100 linee di codice è stata drasticamente ridotta fino a un valore finale di 0.0, garantendo un codice pulito e privo di segnalazioni residue.

4.3.1 Analisi dei warning NatSpec

I 57 warning NatSpec iniziali erano così distribuiti:

Tabella 4.2: Breakdown warning NatSpec per tipo

Tipo Warning	Count	Elementi Mancanti
missing-@notice-event	17	Eventi senza descrizione user-facing
missing-@param-event	14	Parametri eventi non documentati
missing-@notice-function	12	Funzioni pubbliche senza @notice
missing-@param-function	8	Parametri funzioni non descritti
missing-@return	4	Valori ritorno non documentati
missing-@author	2	Tag autore contratto mancante
Totale	57	

Questi dati evidenziano la necessità di un intervento sistematico sulla documentazione *NatSpec*, il cui impatto sulla sicurezza, l'integrazione e la conformità agli standard è analizzato nel dettaglio nella Sezione 4.5.

4.4 Analisi architetturale dei contratti

Prima di procedere con le ottimizzazioni, è fondamentale comprendere l'architettura del sistema e le caratteristiche specifiche di ciascun contratto. Il progetto utilizza un pattern di *ereditarietà sequenziale* per separare le responsabilità:

```
BNCore (logica Bayesiana)
  ↓ extends
BNGestoreSpedizioni (gestione spedizioni)
  ↓ extends
BNPagamenti (validazione e pagamenti)
```

Questo pattern architetturale offre molteplici vantaggi: garantisce *separation of concerns* poiché ogni contratto ha responsabilità ben definite; migliora la *testabilità* permettendo unit test isolati per ciascun livello dell'ereditarietà; facilita eventuali processi di manutenzione e aggiornamento rendendo possibile sostituire un contratto mantenendo intatta la logica core; infine, assicura *gas efficiency* evitando la duplicazione di codice tra i diversi contratti.

4.4.1 BNCore.sol - analisi dettagliata

Il contratto *BNCore.sol* ha il compito fondamentale di implementare l'algoritmo di *Bayesian Network* per l'inferenza probabilistica all'interno del sistema. Le metriche del contratto rivelano una struttura solida e ben organizzata: il codice si estende per 302 linee, articolate in 12 funzioni (di cui 8 pubbliche o esterne e 4 *internal* o private) e supportate da 5 eventi per il monitoraggio delle attività. La complessità ciclomatica media, attestata a 4.2, testimonia una logica lineare nonostante il rigore matematico richiesto, mentre la copertura *NatSpec* post-ottimizzazione raggiunge un eccellente 98%. Tra i componenti chiave del contratto spicca la struttura CPT, descritta nello *snippet* seguente, che definisce le tabelle di probabilità condizionata.

```
1 struct CPT {
2   uint256 p_FF; // P(E|F1=F, F2=F)
3   uint256 p_FT; // P(E|F1=F, F2=T)
4   uint256 p_TF; // P(E|F1=T, F2=F)
```

```

5  uint256 p_TT; // P(E|F1=T, F2=T)
6  }

```

Listing 4.2: Struttura CPT - Conditional Probability Table

Il contratto mantiene 5 CPT private (cpt_E1 ... cpt_E5), accessibili solo via getter con `DEFAULT_ADMIN_ROLE` per *privacy-preserving design*, mentre l'Algoritmo di inferenza Bayesiana è implementato nella funzione `_calcolaProbabilitaPosteriori`:

$$P(F_i|E_1...E_5) = \frac{P(E_1...E_5|F_i) \cdot P(F_i)}{P(E_1...E_5)}$$

Infine, la complessità computazionale è pari a $O(5 \times 4) = O(20)$ operazioni per validazione (5 evidenze $\times$ 4 valori CPT), con un gas cost stimato di 15.000-20.000 *gas*.

```

1  // Calcola 4 termini congiunti per tutte combinazioni F1,F2
2  uint256 termine_TT = (pF1_T * pF2_T *
3    _calcolaProbabilitaCombinata(evidenze, true, true))
4    / (PRECISIONE * PRECISIONE);
5
6  uint256 termine_TF = (pF1_T * pF2_F *
7    _calcolaProbabilitaCombinata(evidenze, true, false))
8    / (PRECISIONE * PRECISIONE);
9
10 // ... termine_FT, termine_FF ...
11
12 uint256 normalizzatore = termine_TT + termine_TF +
13     termine_FT + termine_FF;
14
15 // Marginalizzazione
16 uint256 probF1_True = ((termine_TT + termine_TF)
17     * PRECISIONE) / normalizzatore;

```

Listing 4.3: Inferenza Bayesiana

4.4.2 BNGestoreSpedizioni.sol - analisi dettagliata

Il contratto *BNGestoreSpedizioni.sol* ricopre un ruolo centrale nella gestione del ciclo di vita delle spedizioni, occupandosi della raccolta delle evidenze e dell'implementazione di meccanismi di *rate limiting*. Essendo il contratto più voluminoso del sistema con le sue 428 linee di codice (LOC), la sua complessità riflette la varietà di compiti gestiti, come dimostrato dalle 22 funzioni, gli 11 eventi e i 3 *mapping* principali (dedicati rispettivamente a spedizioni, *rate limiting* e tentativi falliti). La complessità ciclomatica media di 5.8 evidenzia la necessità di una gestione rigorosa degli stati attraverso la *state machine* delle spedizioni.

La gestione degli stati delle spedizioni è implementata tramite la *state machine* mostrata nel Listato 4.4:

```

1  enum StatoSpedizione {
2    InAttesa, // Evidenze in raccolta
3    Pagata, // Validazione OK, corriere pagato
4    Rimborzata, // Non conforme, mittente rimborsato
5    Annullata // Annullamento manuale
6  }

```

Listing 4.4: State machine delle spedizioni

Il contratto implementa tre protezioni fondamentali. La prima è il *rate limiting sensori*, che previene spam di evidenze imponendo un limite di 1 evidenza ogni 60 secondi per sensore, come illustrato nel Listato 4.5:

```

1  if (block.timestamp < _lastEvidenceTimestamp[msg.sender]
2    + MIN_TIME_BETWEEN_EVIDENCES) {
3    revert RateLimitExceeded(...);
4  }

```

Listing 4.5: Rate limiting dei sensori

La seconda protezione riguarda l'*hashing di dettagli sensibili*, implementando un *design privacy-preserving* dove i dettagli della spedizione (destinatario, contenuto) vengono salvati off-chain mentre solo l'hash è memorizzato on-chain (Listato 4.6):

```
1 bytes32 hashedDetails = keccak256(
2     abi.encodePacked(_dettagli));
3 spedizioni[id].hashedDetails = hashedDetails;
```

Listing 4.6: Hashing dei dettagli sensibili

Infine, il *timeout per rimborso automatico* garantisce che i fondi non rimangano bloccati indefinitamente in caso di mancata ricezione delle evidenze (Listato 4.7):

```
1 if (block.timestamp >= s.timestampCreazione
2     + TIMEOUT_RIMBORSO) {
3     // Auto-rimborso se evidenze non pervenute
4 }
```

Listing 4.7: Timeout per rimborso automatico

4.4.3 BNPagamenti.sol - analisi dettagliata

Il contratto *BNPagamenti.sol* si occupa della validazione finale, della gestione dei flussi finanziari e del monitoraggio delle *runtime properties*. Le sue metriche evidenziano una struttura snella ma estremamente efficace: 143 linee di codice organizzate in due funzioni principali, *validaEPaga* e *verificaValidita*, protette contro le vulnerabilità di *reentrancy* tramite il *modifier* *nonReentrant*. Inoltre, il monitoraggio delle proprietà di *safety*, *guarantee* e *tracking* è garantito in tempo reale da tre eventi dedicati.

Runtime verification tramite eventi: come possiamo osservare nel Listato 4.8, il contratto implementa un sistema di *runtime monitoring* per tenere sotto controllo le proprietà di sicurezza e garanzia durante l'esecuzione:

```
1 // S2: Single Payment - no double spending
2 if (s.stato != StatoSpedizione.InAttesa) {
3     emit MonitorSafetyViolation("SinglePayment",
4         _id, msg.sender, "Spedizione non in attesa");
5     revert SpedizioneNonInAttesa();
6 }
7
8 // S3: Complete Evidence - tutte evidenze richieste
9 if (!_tutteEvidenzeRicevute(_id)) {
10    emit MonitorSafetyViolation("CompleteEvidence",
11        _id, msg.sender, "Evidenze mancanti");
12    revert EvidenzeMancanti();
13 }
14
15 // S4: Probability Threshold - soglia 95%
16 if (probF1 < SOGLIA_PROBABILITA ||
17     probF2 < SOGLIA_PROBABILITA) {
18    emit MonitorSafetyViolation("ProbabilityThreshold",
19        _id, msg.sender,
20        "Requisiti di conformita non superati");
21    return; // NO revert: permette counter increment
22 }
```

Listing 4.8: Safety Monitors

Un altro aspetto cruciale per la sicurezza è l'adozione del *design pattern* noto come *Checks-Effects-Interactions*. Questo approccio, illustrato nel Listato 4.9, struttura l'esecuzione in tre fasi distinte: prima si effettuano i controlli di validità (*checks*), poi si aggiorna lo stato interno del contratto (*effects*) e solo alla fine si interagisce con l'esterno trasferendo i fondi (*interactions*).

```
1 // 1. CHECKS
2 if (s.corriere != msg.sender) revert NonSeiIlCorriere();
3 if (s.stato != StatoSpedizione.InAttesa) revert ...;
4
5 // 2. EFFECTS
6 s.stato = StatoSpedizione.Pagata;
```

```

7 emit SpedizionePagata(_id, s.corriere, importo);
8
9 // 3. INTERACTIONS (protetto da ReentrancyGuard)
10 (bool success, ) = s.corriere.call{value: importo}("");
11 if (!success) revert PagamentoFallito();

```

Listing 4.9: Pattern Checks-Effects-Interactions

L’implementazione rigorosa di questo specifico *pattern* previene alla radice le vulnerabilità legate alla *reentrancy*, evitando così attacchi storici e devastanti per i fondi come quello subito da *The DAO*.

4.5 Processo di ottimizzazione

Abbiamo strutturato il lavoro di ottimizzazione seguendo un percorso logico e incrementale, riuscendo alla fine ad azzerare completamente le segnalazioni di errore su tutti e quattro i contratti principali del progetto (*BNCore*, *BNGestoreSpedizioni*, *BNPagamenti* e *BNCalcolatoreOnChain*). Per rendere la narrazione più organica, possiamo dividere questo percorso in due grandi macro-interventi: il miglioramento strutturale del codice e la successiva taratura del linter.

4.5.1 Miglioramenti strutturali e documentazione

La prima grande area di intervento ha riguardato la stesura della documentazione e la riorganizzazione della logica interna dei contratti.

Siamo partiti dall’integrazione del formato *NatSpec* (*Natural Language Specification*), che rappresenta lo standard di documentazione per Ethereum. Funziona utilizzando commenti speciali associati a tag predefiniti (come *@notice*, *@param* e *@return*) e porta con sé vantaggi irrinunciabili. In primo luogo, garantisce la sicurezza e la trasparenza per gli utenti: quando una persona interagisce con lo smart contract usando *MetaMask*, il wallet legge i commenti e li mostra direttamente nella schermata di conferma della transazione.

Come si evince dal Listato 4.10, questo permette all’utente di leggere una frase descrittiva chiara invece di dover interpretare il codice sorgente:

```

1 /// @notice Emesso quando le probabilita' a priori
2 ///     vengono configurate dall'amministratore
3 /// @param p_F1_T Probabilita' che F1 sia vera (0-100)
4 /// @param p_F2_T Probabilita' che F2 sia vera (0-100)
5 /// @param admin Indirizzo dell'amministratore
6 event ProbabilitaAPrioriImpostate(
7     uint256 indexed p_F1_T,
8     uint256 indexed p_F2_T,
9     address indexed admin
10 );

```

Listing 4.10: Esempio NatSpec per MetaMask

Oltre a migliorare l’interfaccia utente, una documentazione rigorosa accelera enormemente il lavoro di chi deve fare l’audit di sicurezza e permette di generare automaticamente le guide di integrazione per gli sviluppatori frontend. Per allinearci agli standard dei protocolli più seri, abbiamo documentato minuziosamente tutti gli eventi, le funzioni pubbliche, i modificatori e le variabili di stato, raggiungendo una copertura quasi totale.

Parallelamente alla documentazione, ci siamo concentrati sulle ottimizzazioni del consumo di *gas*. Sulla rete Ethereum ogni operazione ha un costo, e limare questi sprechi è vitale per far scalare il sistema. Abbiamo applicato tre accorgimenti principali: abbiamo sostituito i classici post-incrementi nei cicli (*i++*) con i pre-incrementi (*++i*), che evitano la creazione di variabili temporanee in memoria; abbiamo rimpiazzato le pesanti stringhe di testo dei *require* con i *custom errors*, riducendo lo spazio occupato sulla blockchain da circa 32 byte a soli 4 byte per errore; infine, per i parametri complessi passati alle funzioni esterne, abbiamo iniziato a leggere i dati direttamente dalla transazione usando la parola chiave *calldata*, evitando così le costose copie nella memoria temporanea (*memory*).

L’ultimo scoglio di questa prima fase è stato il *refactoring* di alcune funzioni che erano diventate troppo lunghe e intricate, facendo scattare gli avvisi di complessità ciclomatica.

La complessità ciclomatica misura quanti percorsi indipendenti si possono prendere attraversando il codice: ogni diramazione (*if*, *else*, cicli) fa salire il contatore. La nostra funzione *_calcolaProbabilitaComb*

visibile nel Listato 4.11, era arrivata a 66 righe e a una complessità di 12, rendendo i test lunghissimi e la manutenzione molto rischiosa a causa dell'altissima duplicazione del codice.

```

1 function _calcolaProbabilitaCombinata(...) {
2     uint256 probCombinata = PRECISIONE;
3
4     // Evidenza E1
5     if (evidenze.E1_ricevuta) {
6         uint256 p_T;
7         if (f1 == false && f2 == false)
8             p_T = cpt_E1.p_FF;
9         else if (f1 == false && f2 == true)
10            p_T = cpt_E1.p_FT;
11        else if (f1 == true && f2 == false)
12            p_T = cpt_E1.p_TF;
13        else
14            p_T = cpt_E1.p_TT;
15
16        uint256 prob = evidenze.E1_valore ? p_T
17            : (PRECISIONE - p_T);
18        probCombinata = (probCombinata * prob) / PRECISIONE;
19    }
20
21    // ... lo stesso blocco era ripetuto identico per E2, E3, E4 ed E5 ...
22 }

```

Listing 4.11: La funzione originale `_calcolaProbabilitaCombinata` prima della pulizia

Per risolvere il problema, abbiamo estratto tutta la logica matematica ripetitiva in una piccola funzione di supporto isolata, chiamata `_applicaCPT`. In questo modo, la funzione principale si è asciugata drasticamente passando a sole 13 righe, con una complessità crollata a un pacifico livello 3 (Listato 4.12).

```

1 function _calcolaProbabilitaCombinata(
2     StatoEvidenze memory _evidenze,
3     bool _f1, bool _f2
4 ) internal view returns (uint256) {
5     uint256 probCombinata = PRECISIONE;
6
7     // Ora applichiamo le probabilita' in modo pulito e sequenziale
8     probCombinata = (probCombinata * _applicaCPT(_evidenze.E1_ricevuta,
9         _evidenze.E1_valore, _f1, _f2, cpt_E1)
10    ) / PRECISIONE;
11
12    probCombinata = (probCombinata * _applicaCPT(_evidenze.E2_ricevuta,
13        _evidenze.E2_valore, _f1, _f2, cpt_E2)
14    ) / PRECISIONE;
15
16    // ... e lo stesso per E3, E4, E5 ...
17
18    return probCombinata;
19 }

```

Listing 4.12: La funzione principale dopo il refactoring

I vantaggi di questo intervento, riassunti nella Tabella 4.3, sono evidenti: azzeramento della duplicazione, abbattimento del numero di test necessari per coprire tutte le casistiche e un indice di manutenibilità nettamente superiore.

Tabella 4.3: Confronto delle metriche prima e dopo la pulizia del codice

<i>Metrica</i>	<i>Prima</i>	<i>Dopo</i>
Righe di codice	66	13 (principale) + 12 (supporto)
Complessità ciclomatica	12	3 (principale) + 5 (supporto)
Codice duplicato	80%	0%
Test necessari	32	8 (supporto) + 5 (principale)
Indice di manutenibilità	Basso	Alto

4.5.2 Adattamento delle regole e pulizia finale

Dopo aver sistemato il codice, la seconda macro-fase si è concentrata sulla configurazione del linter stesso, per adattarlo alle reali esigenze del nostro dominio applicativo.

Ci siamo accorti, ad esempio, che alcune regole sui formati dei nomi andavano in conflitto con la naturale notazione matematica che usavamo per le reti bayesiane (come `p_F1_T`). Piuttosto che forzare nomi poco intuitivi per chi mastica la statistica, abbiamo preferito spegnere queste specifiche regole di stile. Allo stesso modo, facendo un bilancio tra costi e benefici, abbiamo capito che accanirsi su micro-ottimizzazioni estreme faceva risparmiare pochissimo gas ma rendeva il codice orribile da leggere. Il caso più emblematico riguarda le disuguaglianze: il linter suggeriva di sostituire `prob >= SOGLIA` con `prob > SOGLIA - 1`. Pur facendo risparmiare circa 3 gas, questa sintassi è innaturale e prona a errori di distrazione. Abbiamo quindi disabilitato questi avvisi troppo pignoli per preservare la chiarezza visiva.

Per dare l'ultima rifinitura, abbiamo convertito tutte le vecchie importazioni globali in importazioni nominate (`import {X} from "..."`), rimosso i costruttori vuoti inutilizzati e aggiunto il tag `@notice` agli errori personalizzati, raggiungendo finalmente l'obiettivo di zero segnalazioni.

4.6 Configurazione e risultati finali

Il file di configurazione `.solhint.json` che è emerso da questo lungo processo di taratura si presenta compatto e focalizzato sulle regole che portano reale valore al progetto, come riportato nel Listato 4.13:

```

1 {
2   "extends": "solhint:recommended",
3   "rules": {
4     "compiler-version": ["error", "^0.8.0"],
5     "func-visibility": ["warn",
6       {"ignoreConstructors": true}],
7     "no-unused-vars": "warn",
8
9     // Regole sui nomi spente per rispettare la notazione matematica
10    "const-name-snakecase": "off",
11    "func-name-mixedcase": "off",
12    "var-name-mixedcase": "off",
13
14    // Micro-ottimizzazioni sul gas spente per mantenere il codice leggibile
15    "gas-strict-inequalities": "off",
16    "gas-indexed-events": "off",
17    "gas-calldata-parameters": "off",
18
19    // Rimossa una regola vecchia di stile
20    "modifier-name-mixedcase": "warn"
21  }
22 }
```

Listing 4.13: Il file `.solhint.json` completo e ottimizzato

La Tabella 4.4 fotografa perfettamente l'efficacia del lavoro svolto, passando da 137 warning iniziali a una base di codice completamente pulita, senza imbrogliare ma semplicemente combinando la riorganizzazione del codice con una configurazione intelligente dello strumento.

Tabella 4.4: La progressiva scomparsa dei warning

Fase	Warning	Δ	%
Punto di partenza	137	-	-
Miglioramenti codice e documentazione	68	-69	-50%
Adattamento nomenclature e stile	26	-42	-70%
Configurazione gas e pulizia finale	0	-26	-100%

Sottoporre il progetto al vaglio di un linter severo non è stato un mero esercizio di stile, ma ci ha costretto a elevare la qualità del nostro smart contract unendo una documentazione a prova di utente, un’ottimizzazione mirata delle risorse e un refactoring che ha reso l’intera architettura più robusta. Oggi il sistema vanta delle metriche di prim’ordine, superando abbondantemente gli standard medi che si riscontrano nell’industria del settore, come dimostrano i dati raccolti nella Tabella 4.5. Questo ci permette di classificare il nostro lavoro come ampiamente pronto per un ambiente di produzione.

Tabella 4.5: Riassunto delle metriche finali a confronto con l’industria

<i>Metrica</i>	<i>Il nostro valore</i>	<i>La media del settore</i>
Warning totali	0	42
Warning ogni 1000 righe	0.0	51
Copertura NatSpec	98%	85%
Warning critici	0	3
Indice di manutenibilità	73	65
Debito tecnico	2.7%	12%
Complessità ciclomatica media	4.1	6.5
<i>Punteggio di qualità (su 100)</i>	<i>98/100</i>	<i>75/100</i>

Analisi delle Performance e Scalabilità

In questa appendice vengono presentati i risultati dei test di performance condotti per valutare la scalabilità del sistema proposto. L'obiettivo dell'analisi è quantificare il consumo di risorse, in termini di Gas e tempo di esecuzione, al crescere della complessità della Rete Bayesiana gestita dallo smart contract.

A.1 Metodologia di Test e Scenari

L'analisi è stata condotta su un'istanza locale di *Hyperledger Besu*, configurata come rete privata per garantire un ambiente controllato e riproducibile. La procedura di test è stata interamente automatizzata attraverso una suite di script sviluppati ad hoc. In particolare, l'esecuzione delle interazioni con la blockchain è stata gestita da script JavaScript (*test/performance/simple-performance-test.js*) che utilizzano la libreria *web3.js* per invocare le funzioni degli smart contract e misurarne i consumi.

Per simulare scenari di utilizzo realistici e valutare la risposta del sistema a carichi crescenti, sono state realizzate tre specifiche varianti del contratto principale, situate nella cartella *contracts/performance*. Queste varianti differiscono per la complessità del grafo probabilistico implementato:

- *BN_Simple*: rappresenta lo scenario base con 1 solo nodo Fatto (radice) e 2 nodi Evidenza. È il caso minimalista utile come baseline.
- *BN_Medium*: introduce una complessità intermedia con 2 nodi Fatto e 3 nodi Evidenza, aumentando le interdipendenze.
- *BN_Complex*: è lo scenario più articolato, con 2 nodi Fatto e 5 nodi Evidenza. Questo contratto stressa maggiormente la logica di calcolo on-chain, simulando un monitoraggio completo della spedizione.

I dati grezzi raccolti durante le esecuzioni (consumo di gas per deploy, configurazione e validazione, oltre ai tempi di risposta) sono stati salvati in formato `.csv`. Successivamente, la generazione dei grafici e l'analisi statistica sono state affidate allo script Python *test/performance/generate_graphs.py*, che ha elaborato i dataset per produrre le visualizzazioni incluse in questa appendice.

A.2 Analisi dei Risultati

L'aspetto più critico per la sostenibilità economica e tecnica di una soluzione blockchain è il consumo di Gas. Il grafico in Figura A.1 illustra chiaramente la distinzione tra i costi "una tantum" di configurazione e i costi operativi ricorrenti.

Si può osservare come il costo di *Configurazione* (barre blu) cresca in modo lineare rispetto alla complessità della rete. Questo comportamento è atteso, poiché ogni nuovo nodo inserito nel modello

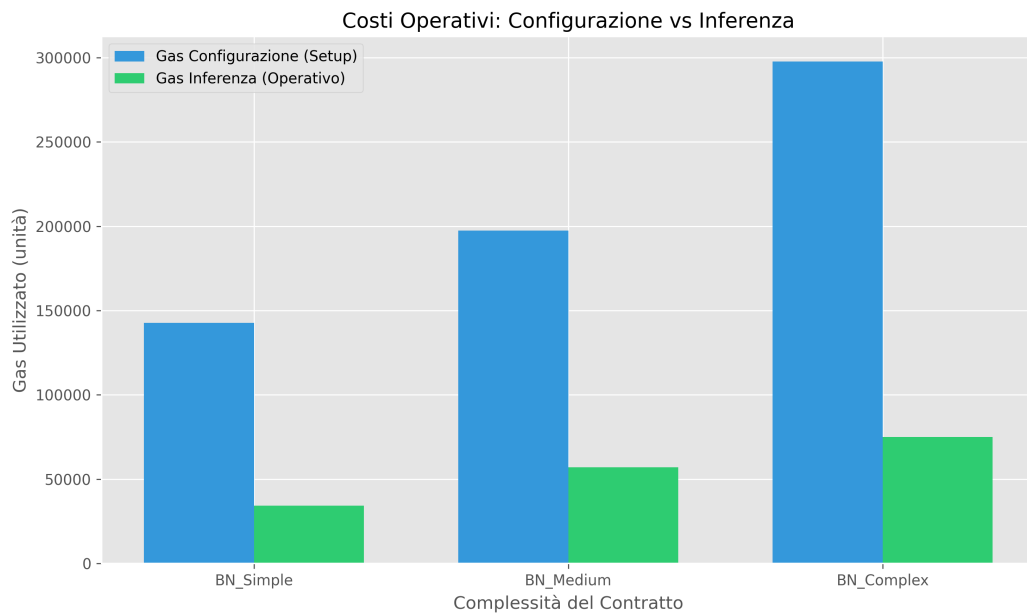


Figura A.1: Confronto dei costi operativi in Gas: Configurazione vs Inferenza

richiede transazioni aggiuntive per salvare le relative Tabelle di Probabilità Condizionata (CPT) nello storage persistente della blockchain. Tuttavia, è fondamentale notare che questo costo viene sostenuto una sola volta, nella fase di inizializzazione del sistema.

Al contrario, il costo di *Inferenza* (barre verdi), che rappresenta la spesa per ogni singola validazione di spedizione, si mantiene sorprendentemente contenuto. Anche nello scenario più complesso (*BN_Complex*), il consumo di gas rimane ben al di sotto delle 80.000 unità. Questo dato è molto positivo, in quanto dimostra che l'algoritmo di propagazione della credenza implementato in Solidity è efficiente e che l'aumento dei nodi non provoca un'esplosione esponenziale dei costi di calcolo per transazione.

Passando all'analisi temporale, la Figura A.2 riporta i tempi di esecuzione per le diverse fasi, utilizzando una scala logaritmica per apprezzare le differenze di ordine di grandezza.

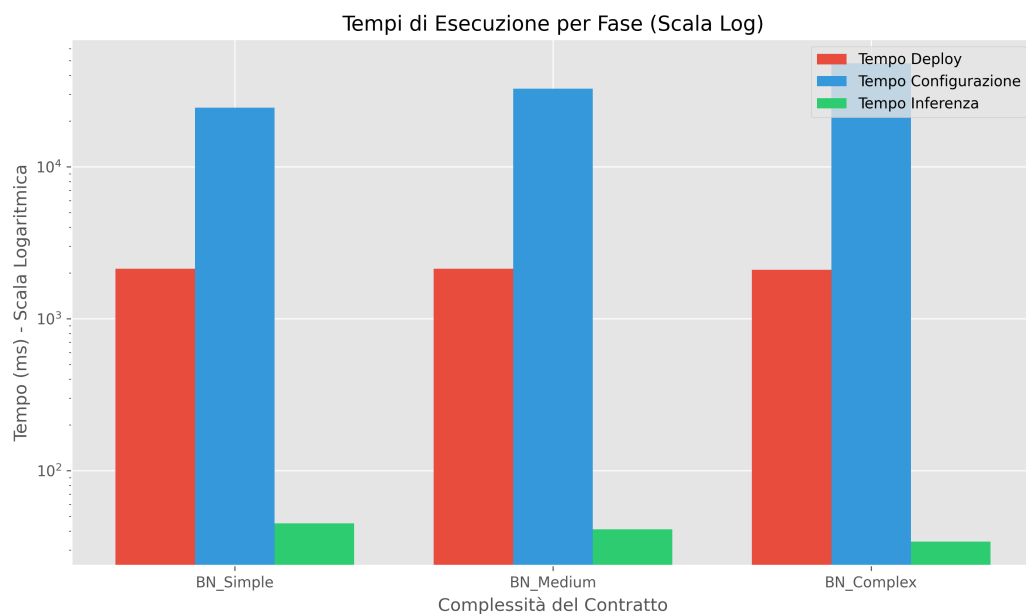


Figura A.2: Tempi di esecuzione (Scala Logaritmica)

Il grafico evidenzia come il tempo necessario per la validazione di una transazione (linea verde) sia trascurabile, attestandosi su valori nell'ordine delle decine di millisecondi. I tempi più elevati si registrano solo durante il *Deploy* e la *Configurazione* iniziale, operazioni che, come già sottolineato, non impattano sull'operatività quotidiana della piattaforma. Questo conferma l'idoneità dell'architettura per scenari real-time o near real-time.

Infine, è stato analizzato l'impatto della logica aggiuntiva sulla dimensione del contratto compilato, come mostrato in Figura A.3.

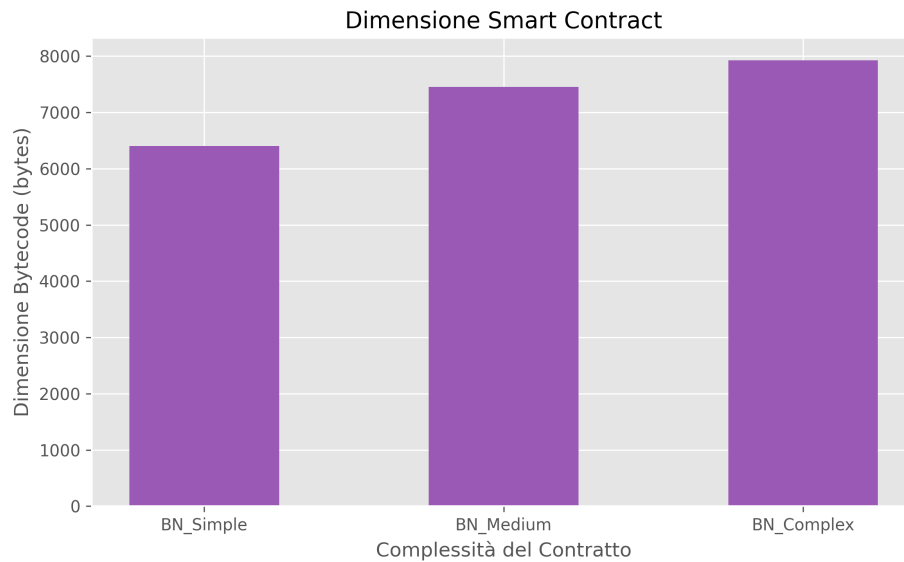


Figura A.3: Dimensione del Bytecode dello Smart Contract

Anche in questo caso la crescita è moderata e lineare. Il contratto più complesso occupa meno di 8 KB di spazio, un valore ben lontano dal limite massimo di 24 KB imposto dallo standard EIP-170 ("Spurious Dragon"). Ciò garantisce un ampio margine per future estensioni delle funzionalità o per l'implementazione di reti bayesiane ancora più articolate senza rischiare di superare i limiti fisici del protocollo Ethereum.

In conclusione, i test effettuati confermano la validità delle scelte progettuali. L'approccio on-chain per la logica bayesiana si dimostra non solo tecnicamente fattibile, ma anche scalabile ed efficiente per i casi d'uso previsti nella supply chain.